Web Search

Multimedia Retrieval · Chapter 8

Multimedia Retrieval - 2026 - Uni Basel

Roger Weber

Contents

8 - Web Search **1**
The Web Search Landscape 3
How Web Search Differs from Classical Retrieval 3
What Users Search For 4
From Intent to Result Page 5
Crawling the Web 8
Ranking Signals Beyond Term Counts 10
Term Proximity 10
Field Weighting with BM25F 12
Anchor Text 13
Location 14
Language 14
Quality, Spam, and Trust 15
Link Analysis: PageRank and HITS 16
Counting Links Is Not Enough 16
The Random Surfer 16
Computing PageRank by Power Iteration 17
Personalized PageRank 19
Hubs and Authorities: HITS 19
Learning to Rank 23
The Ranking Stack 23
Combining Signals by Hand 23
The Learning-to-Rank Problem 24
Learning to Rank in Practice 29
Ranking Without a Query: Feeds 31
Summary 33
Model Comparison 33
Key Takeaways 33
Key Formulas 34
Self-Check Questions 34
Further Reading 35

8 - Web Search

Type “uni basel” into a search engine. Even two short words are a strong filter: of the hundreds of billions of pages in the index, almost all contain neither term and drop out. But a tiny fraction of the web is still thousands to millions of pages: the university’s home page, every course and news page, student blogs, a marathon result list with a runner from Basel. The ranking functions of the earlier chapters score these pages by how well their words match the query, and for a two-word query most of them match about equally well. Yet the user expects the home page at rank 1, and expects it within a fraction of a second. The words of the query cannot decide this ranking on their own.

Four problems make the web the hardest setting for retrieval in this book:

- **Too many equal matches.** A short query discards almost the whole web but still leaves far more matching pages than anyone can read, with nearly equal term-based scores.
- **Different intents, one result list.** Queries range from a site name such as “instagram”, typed instead of a URL, to a full question such as “Who was Albert Einstein?”. Each intent expects a different kind of result (Intent Routing and Classification). With two or three words and no context, the engine cannot know what one particular user wants, so it serves what most users want. For “apple”, that is the home page of Apple, not an article about the fruit.
- **No editor.** The collections of earlier chapters were curated: nobody adds hundreds of useless books to a university library catalogue to promote their own. The web has no such gatekeeper. Its content ranges from careful reference works to outdated or false information, clickbait, and scams, and much of it exists in many copies, such as mirrored manuals, the documentation of popular frameworks, and copied blog posts. The first web search engines learned the hard way that term matching ranks such pages high.
- **Adversarial ranking.** A position on the first result page is worth money, because results on the second page are almost never clicked. Search engine optimization (SEO) is an industry that tunes pages to rank higher, from legitimate improvements to plain manipulation. Unlike in a library, no central authority decides which document deserves the top position, and every signal that rewards a page invites someone to fake it.

If the keywords cannot decide, the engine needs evidence from elsewhere. The web offers more of it than any curated collection:

- **The query intent.** The engine detects the type of query and applies a strategy for it. A navigational query must lead to the official website every time; for frequent queries, this can come from a curated list, much like the entries of an FAQ service.
- **The page and what others say about it.** Term positions, the HTML structure, and the anchor texts of the links that point to a page (Ranking Signals Beyond Term Counts).
- **The link graph.** Popular pages receive more links, so the link structure shows what people in general consider important (Link Analysis: PageRank and HITS). The goal is not to optimize every single query, but to make the great majority of the queries people actually type work very well.
- **User behaviour.** Clicks, the time users spend on a result, and reformulated queries show which results satisfy. Learning to rank combines hundreds of such signals into one model that keeps adapting as behaviour changes (Learning to Rank).

The intent also decides what the result page looks like. Instead of one list of links, the engine sends the query to specialized searches and composes the page for it: a highlighted passage or a generated answer for a question, facts from a curated knowledge base for a known entity, a map for a place, a product list for a purchase, and related searches for the next step. All of this rests on a crawler that does more than copy the web: by following its links, it collects the evidence about each page that the page itself does not provide (The Web Search Landscape).

The same machinery now works without a query at all. Social media feeds and video platforms decide, for every user and every scroll, which post to show next. They retrieve and rank candidates just like a search engine, but the user's history plays the role of the query, and the reactions of communities and individual users train a ranking that is personalized for every user (Ranking Without a Query: Feeds).

What you'll learn

- Explain why term-based ranking breaks down on the web (equal matches, diverse intents, uncurated quality, SEO), how the query intent shapes the result page, and what a crawler contributes beyond collecting pages.
- Compute web-specific content signals: term proximity, BM25F field weighting, and per-domain anchor-text scores.
- Compute PageRank by power iteration and HITS hub and authority values, and contrast global with query-specific link analysis.
- Compare pointwise, pairwise, and listwise learning to rank, and explain how click data must be debiased before training.
- Explain how a feed ranks content without a query, and why optimizing engagement differs from optimizing value.

Prerequisites

- Classical Text Retrieval: the vector space model and BM25, which web signals extend (see BM25).
- Performance Evaluation: graded relevance and nDCG, the target measure of learning to rank (see Evaluating Graded Relevance).
- Index for Text Retrieval: inverted files and postings lists, which proximity scoring extends with term positions.
- Semantic Search and Vector Search: embeddings, cross-encoders, and approximate nearest-neighbour search, reused for neural reranking and feed retrieval (see Semantic Similarity with Transformers).
- Retrieval Augmented Generation: generated answers, which modern result pages show above the links.

The Web Search Landscape

The chapter opener named four problems: too many equal matches, diverse intents, no editor, and adversarial ranking. This section looks at them more closely and then at the two ends of a web search engine: the result page, which the engine composes for the intent of each query, and the crawler, which collects the pages and the evidence about them.

How Web Search Differs from Classical Retrieval

Five differences shape every design decision in web search:

aspect	classical retrieval	web search
collection	controlled and quality-assured, such as a product catalogue or a library	no quality control; ranges from reference works to errors, propaganda, and spam
documents	shared structure, rich metadata, structured or faceted search	heterogeneous formats (HTML, PDF, scripts); beyond title, URL, and text, little reliable structure
queries	narrow context, such as artists, albums, and songs in a music database	any topic, two or three words, no explicit context; the engine must infer the intent
results	manageable result sizes, filters, facets, sorting	billions of matching pages for a short query; ranking decides everything
relationships	curated relations with little extra information	a huge link graph whose topology reveals which pages people consider important

Classical retrieval assumes that every document in the collection deserves to be found and that its text describes it honestly. On the web, neither assumption holds.

Much low-quality content is not malicious. Pages that were correct when written become outdated, blogs are abandoned, and errors are never corrected. Other content is deliberately misleading: false information, scams that imitate banks or shops, and clickbait whose content does not deliver what its title promises. On top of this, much of the web exists in many copies. The manuals of popular products, the documentation of popular frameworks, and successful blog posts are mirrored and republished on many sites, and the same article is often reachable under several URLs. A ranking that only matches terms scores all copies equally, so the result list may show the same content ten times. In a library, a librarian filters all of this before a document enters the collection. On the web, the search engine must do it at ranking time, with the quality and spam signals of Ranking Signals Beyond Term Counts.

Ranking on the web is also adversarial, because visibility is worth money. In a backlinko.com analysis of 4 million Google search results, the first organic result received an average click-through rate of 27.6%, and only 0.63% of searchers clicked on anything on the second page. A page that is not on the first page is practically invisible. **Search engine optimization** (SEO) is the discipline of improving a page's position in the results. Much of it is legitimate and welcome to search engines, which publish guidelines for it: descriptive titles, a clean HTML structure, fast pages, and good content that others link to. Its other side manipulates the ranking signals directly, with keyword stuffing, hidden text, link farms, and cloaking, which serves the crawler different content than the user. No central authority decides which page deserves the top position. The ranking itself is the only arbiter, and many of the signals in this chapter were designed to be hard to fake.

Together, these differences shift the weight of the ranking away from the page text. A two-word query matches millions of pages that all look equally similar to the query, and the page text is exactly what an SEO controls. Signals that do not come from the page text, such as links, anchors, and user behaviour, therefore carry much of the ranking.

Parts of the web are invisible to a search engine. Pages behind logins, results of database queries, and content that single-page applications assemble in the browser with JavaScript are publicly reachable but

hard or impossible to crawl. This **deep web** is many times larger than the indexed web. A crawler can reach a deep page only if some other page links to it directly, a so-called deep link.

What Users Search For

Andrei Broder classified web queries by the user's goal into three types:

- **Navigational** queries aim at one specific website or a section of it: "youtube", "amazon prime", "facebook login". Bookmarks were designed for this need, but many users prefer to type the name into a search engine. There is usually exactly one correct result.
- **Transactional** queries express the intent to do something, typically to buy, book, or download: "new coffee machine", "trip to Hawaii". The transactional intent is often implicit.
- **Informational** queries seek knowledge about a topic: news, events, people, how-to instructions. Some are phrased as questions: "who won the 2025 Nobel Prize in Physics", "how to make mac and cheese", "where to swim near Basel".

The intent changes what a good result looks like. A navigational query is answered by one link at rank 1. An informational question may be answered best by a sentence on the result page itself, and a transactional query by a list of products to compare. Chapter 3 shows how a system detects the intent of a query and routes it to a backend. Broder's three types are a coarse version of the routing intents there. A navigational query stays in the web index, with the strong expectation of a single result. Informational and transactional queries are often routed further, for example "Who was Albert Einstein?" to a people search, "where to swim near Basel" to a map search, and "new coffee machine" to a product search.

Most queries do not state their intent, and many are ambiguous. "apple" can mean the company, the fruit, or the record label; "jaguar" can mean the car or the animal. The engine cannot ask back, so it ranks for the interpretation that most users mean, which it learns from the clicks of earlier users. For "apple", the home page of Apple comes first. Rarer interpretations receive a few positions further down, so that a user with a less common intent still finds something on the first page. This mix of interpretations is called **result diversification**.

Frequent navigational queries need no ranking computation at all. The correct target of "instagram" or "uni basel" is known and stable, and the engine can serve it from a curated or learned table of query-target pairs, much like the entries of an FAQ service. Here, a single wrong result is a failure: if the official site is not at rank 1, the quality of the remaining ranks does not matter.

Web queries are short. Figure 8.1 shows the share of search volume by query length: most search volume comes from queries of one to three words and around ten to twenty characters. Longer queries are more frequent among the *distinct* query strings than in the volume, because each long query is typed by few users.

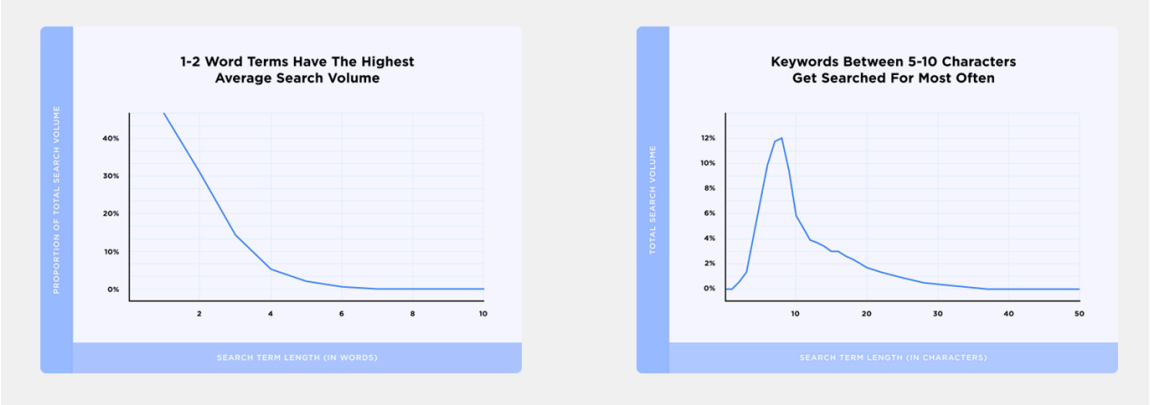


Figure 8.1: Share of search volume by query length in words (left) and characters (right). Source: backlinko.com.

Query frequencies follow Zipf's law. A small set of queries, mostly navigational ones such as "amazon" or "netflix", is repeated millions of times per month and forms the **fat head** of the search demand curve in Figure 8.2. In the **chunky middle**, queries reach thousands of searches per month. The **long tail** contains all the rest, each typed fewer than about a hundred times per month. In a 2020 analysis of 306 million keywords, backlinko.com found that 91.8% of all queries are long-tail queries with at most 100 searches per

month, and that the median query is searched only 10 times per month. The head nevertheless carries a large share of the total volume: the 500 most popular queries alone account for 8.4% of all searches.

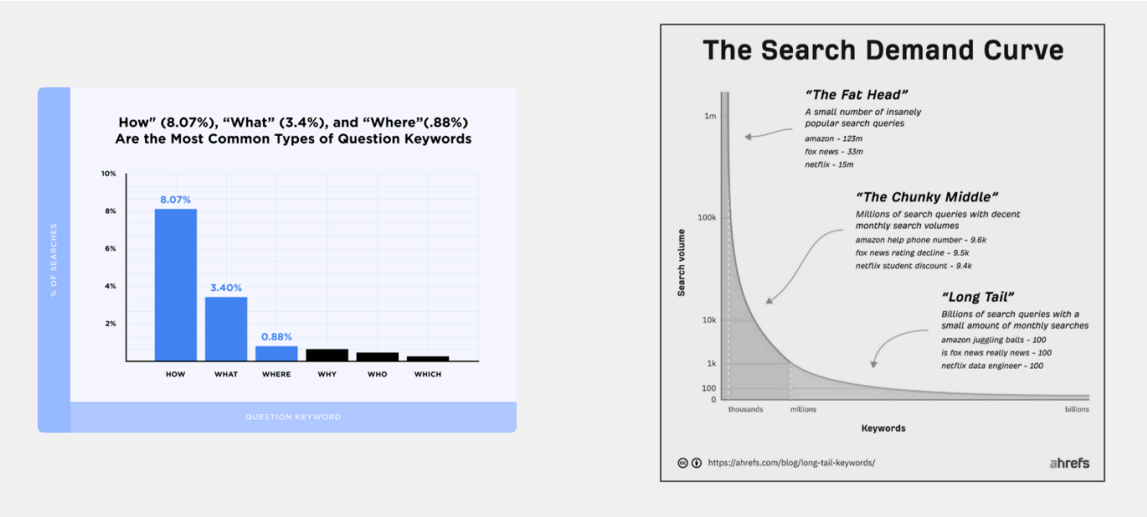


Figure 8.2: Share of searches by question word (left) and the search demand curve with its fat head, chunky middle, and long tail (right). Sources: backlinko.com, ahrefs.com.

The long tail is where search engines compete. Head queries are easy: any engine can learn the right answer for "youtube" from the clicks of millions of users, or simply store it. Tail queries have little or no click history, and many are entirely new. Google reports that about 15% of the queries it sees every day have never been seen before, driven by news, new products, and the endless variety of how people phrase a question. About 14% of queries are phrased as questions (backlinko.com); "how" is the most common question word, followed by "what" and "where" (Figure 8.2, left). Many queries without a question word are implicit questions too: a person's name usually means "who is …", and a product name often means "what does it cost".

From Intent to Result Page

Early search engines treated every query the same way. A single retriever over an inverted index returned a list of links with ads in between, the "ten blue links" (Figure 8.3). For a question such as "who is the US president", the user had to open a result, find the answer on the page, and leave the search engine, which could then no longer show them ads.

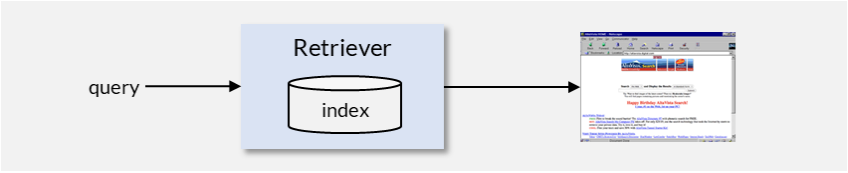


Figure 8.3: Early web search: a retriever over an inverted index returns a list of links.

A modern engine produces the result page in three steps. It detects the intent of the query, sends the query to one or several specialized searches, and generates the elements of the page from their results. Each step keeps more of the user's journey on the result page.

Intent detection and specialized searches. Besides the web index, a search engine runs vertical searches over collections of its own: images, videos, news, maps with local businesses, products with prices, and flights. The intent classifier decides which verticals to query and how to mix their results into the web results. Some intents need no search at all: weather, calculations, unit conversions, and translations are answered directly, like the one-shot vertical intents of Chapter 3.

Ranking and curated knowledge. The web results themselves are ordered by a learned rank model that combines hundreds of signals (Learning to Rank). For entities, the engine adds curated content. In 2012, Google launched its Knowledge Graph, a curated database of entities and facts from reliable sources. When a query names a known entity, a card next to the results shows the key facts directly (Figure 8.4). At launch,

the Knowledge Graph held about 3.5 billion facts on 500 million entities; by 2020, Google reported 500 billion facts on 5 billion entities. Product and brand queries receive similar cards with prices and offers. This works only for facts and products in the provider's knowledge base.

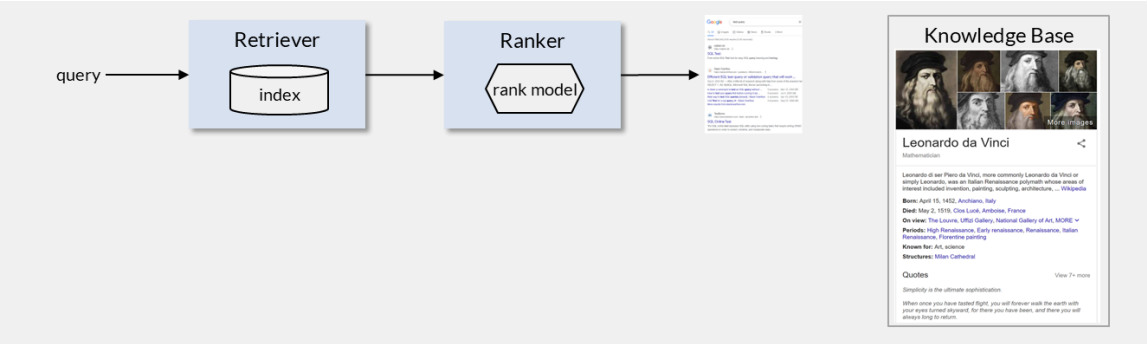


Figure 8.4: Retriever and ranker: a learned rank model orders the candidates; a knowledge panel shows facts about a recognized entity.

Snippets and highlighted passages. Snippets, the short text excerpts below each link, have existed since the early days. In January 2014, Google added **featured snippets**: for question-like queries, a passage extracted from one result is shown above all links. With BERT-based passage ranking, launched in 2021, the engine scores individual passages within long pages. This is the retriever-reader architecture of Chapter 7: a reader model marks the answer span in a retrieved document (Figure 8.5). Most browsers now support text fragments ("scroll to text"), so a click on the result jumps directly to the highlighted passage.

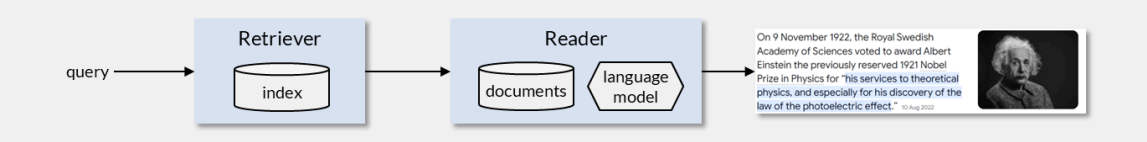


Figure 8.5: Retriever and reader: a language model extracts the answer passage from retrieved documents.

Generated answers. In February 2023, Microsoft added a chat assistant based on OpenAI's GPT-4 to Bing (renamed Copilot later that year), the first major search engine to answer with retrieval-augmented generation (Chapter 7): the engine retrieves top results and a language model writes an answer with links to its sources (Figure 8.6). Google followed with AI Overviews, generated summaries above the results, in 2024 and with a conversational AI Mode in 2025. Generated answers are expensive, so search engines show them selectively, mainly for informational queries.

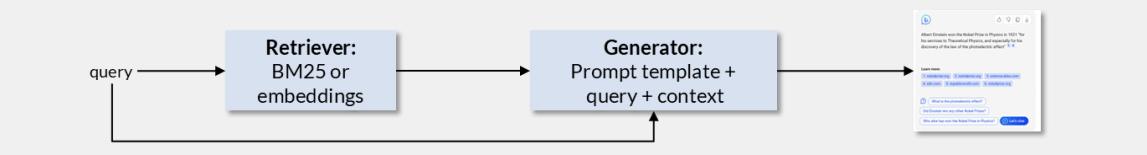


Figure 8.6: Retrieval-augmented generation in web search: a generator writes an answer from the retrieved results.

Related searches. Many searches are one step in a longer task. Related searches and related questions ("People also ask") suggest the next step, derived from the query logs of other users with the same need.

Most of these components appeared within the last fifteen years, on top of a ranking core that dates from the late 1990s.

How web search engines evolved (optional reading)

In the 1990s, the web was small enough to be organized by hand. Yahoo started as a human-edited directory, a hierarchical catalogue of websites sorted into topics, and added keyword search later. AltaVista, launched in 1995, was one of the first engines to index the full text of a large part of the web. Its ranking relied on classical term weighting with some tweaks such as boosts for terms in the title or domain name. With a few million pages, this was good enough: queries matched few pages, and the best ones usually contained the query terms prominently.

Google grew out of a research project by Larry Page and Sergey Brin at Stanford and launched as a company in 1998. Three ideas set it apart. It ranked pages with link analysis (PageRank) and anchor text, which were much harder to manipulate than the page text. It optimized its ranking for the most common types of web queries instead of generic relevance. And it ran on a distributed architecture of cheap commodity machines, which let it crawl more pages and answer more queries than its competitors. Within a few years, Google dominated web search.

The 2000s added machine learning to the ranking (learning to rank, Learning to Rank), universal search that mixes images, news, videos, and maps into the result list, and the Knowledge Graph in 2012. The late 2010s brought transformer models: BERT for understanding queries and ranking results in 2019, and the ranking of individual passages within pages in 2021.

The 2020s brought generated answers. Microsoft's integration of GPT-4 into Bing in 2023, Google's AI Overviews in 2024, and chat-first assistants such as ChatGPT search and Perplexity changed what users expect from a search engine: an answer, not a list. Classical ranking remains at the core of these systems, because the generated answer is only as good as the retrieved results it is based on.

The Assembled Result Page

The result page (SERP, search engine results page) combines the outputs of all these components. Figure 8.7 shows a schematic for "uni basel". An AI overview with source links sits at the top. Ads follow, marked as sponsored and ranked by bid and predicted relevance. The organic results are the ranked list of web pages with title, URL, and snippet; this is what the ranking signals of this chapter decide. Related questions expand to a featured snippet each, a carousel shows images or videos, and the knowledge panel on the side shows facts about the university.

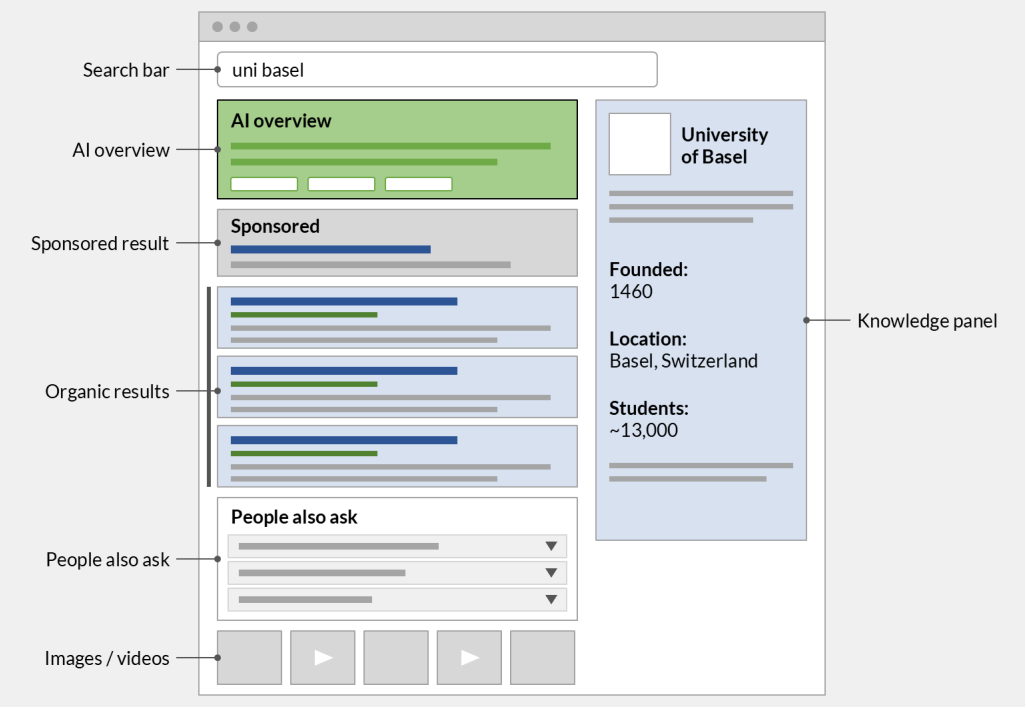


Figure 8.7: Schematic of a modern result page for "uni basel": AI overview, ads, organic results, related questions, media carousel, and knowledge panel.

The search engine must decide for each query which elements to show and in which order. This is itself a ranking problem, with the additional constraint that the page must load in a fraction of a second.

Answers on the result page change the economics of the web. When the answer is on the result page, the user has no reason to click. A 2024 clickstream study by SparkToro found that 58.5% of Google searches in the US and 59.7% in the EU end without a click on any result, a share that has grown with every new answer feature. Publishers, who depend on search traffic, lose visitors, while the search engine keeps users

on its own page. A new optimization industry has emerged as well: **generative engine optimization** (GEO) tries to make a page more likely to be cited in generated answers, for example by adding statistics, quotations, and authoritative sources. Aggarwal et al. reported that such changes can raise a page's visibility in generated answers by up to 40%.

Crawling the Web

A search engine can only rank pages it has fetched. Collecting the pages, however, is the smaller part of what a crawl provides. Sites can already announce their content themselves: a sitemap lists the URLs of a site, and some engines accept notifications of changed pages through protocols such as IndexNow. But everything a site submits is the site's own claim about itself, and the web has no editor and strong incentives to cheat. The crawler gives the engine an independent view of the web, and this view is what most ranking signals of this chapter are built on:

- **The view from outside.** By following links, the crawler sees each page through the rest of the web: which pages link to it, and with which anchor texts. This evidence is written by others and is therefore much harder to fake than the page's own text. It feeds the anchor index (Ranking Signals Beyond Term Counts) and the link graph (Link Analysis: PageRank and HITS). Global measures such as PageRank need the whole graph, which no single site can provide.
- **The content as delivered.** The crawler fetches a page the way a browser does, so the engine judges the content that users actually receive, not the keywords and descriptions that a site declares about itself in its metadata.
- **Duplicates and change.** Only a crawl across many sites reveals that a page is one of a hundred copies, and only repeated visits reveal how often a page changes, which the engine needs to keep its results fresh.
- **The basis for answers.** Featured snippets, knowledge panels, and generated answers are only as good as the crawled pages behind them. The crawl is the corpus that the generated answers of Chapter 7 are grounded on.

The crawler (also called spider or bot) works in the loop shown in Figure 8.8:

1. The crawl starts from a set of **seed URLs**, for example well-known sites and previously crawled pages.
2. The **URL frontier** holds all URLs waiting to be fetched, ordered by priority. Important pages (high PageRank, frequent changes) are fetched first.
3. The **fetcher** downloads a page. Before fetching from a site, it reads the site's `robots.txt` file, which lists the paths the site owner does not want crawled.
4. The **parser** extracts the text, the outgoing links, and their anchor texts. New URLs go back into the frontier; each link and its anchor text is stored under the target URL for the anchor index (Ranking Signals Beyond Term Counts) and the link graph (Link Analysis: PageRank and HITS).
5. **Duplicate detection** discards pages already seen. Exact duplicates are found by hashing the URL and the content; near-duplicates, such as the same article with different ads, by comparing compact fingerprints of the content such as SimHash.

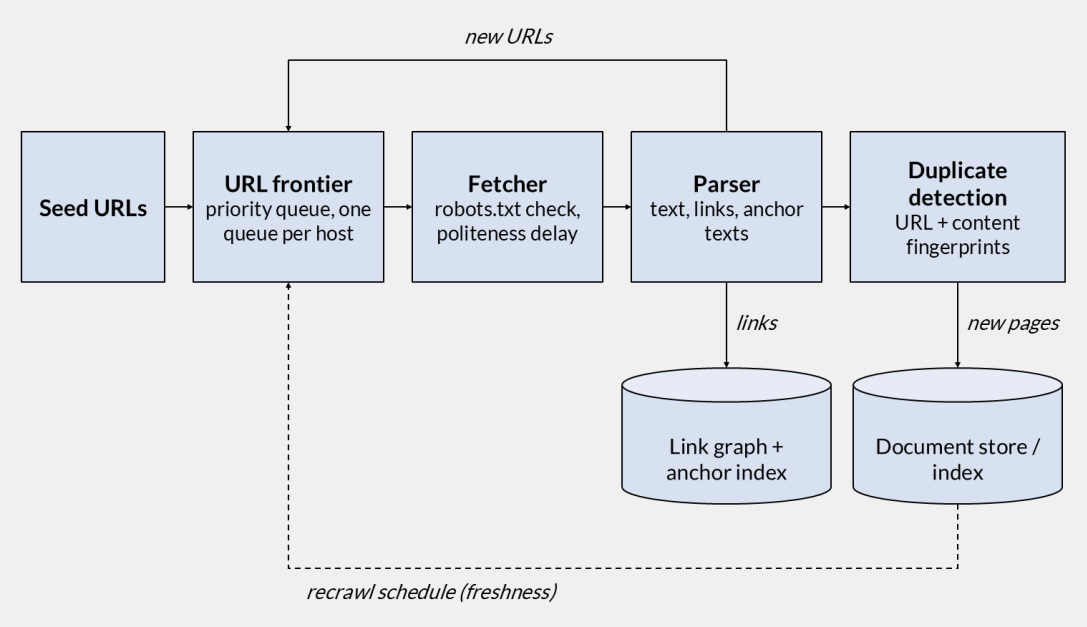


Figure 8.8: The crawler loop: frontier, fetcher, parser, and duplicate detection feed the index and the link graph.

Three requirements make crawling at web scale hard. **Politeness**: a crawler must not overload a site, so it limits the request rate per host, typically by keeping one queue per host in the frontier and waiting between requests. **Freshness**: pages change at very different rates. A news front page changes every few minutes, a professor's home page every few months. The crawler estimates the change rate of each page and schedules recrawls accordingly; the index is never a snapshot of the web, but a mix of pages crawled at different times. **Robustness**: the web contains spider traps, such as calendars with an endless sequence of "next month" links, and servers that respond slowly or with garbage.

The `robots.txt` convention dates from 1994 and was standardized only in 2022 (RFC 9309). It is a request, not an access control: well-behaved crawlers follow it, but nothing enforces it. It has gained new attention with language models. AI companies crawl the web to collect training data, and site owners increasingly use `robots.txt` to block crawlers such as OpenAI's GPTBot while still allowing search engine crawlers.

Ranking Signals Beyond Term Counts

Recall the query “uni basel” from the chapter opener. Figure 8.9 shows two of the many matching pages: a course page that mentions “Universität Basel” in its header (left) and the university home page (right). By term counts alone, both are plausible, and BM25 may even prefer the shorter course page. Classical retrieval treats every occurrence of a term as equal evidence and ignores where it occurs. Web search engines extend the document description with additional signals:

- **proximity** of the query terms in the document (the closer, the better)
- **field weighting**: occurrences in the title or a heading count more than occurrences in body text
- **anchor text**: the words that other pages use when they link to this page
- **location** of the user and the page
- **language** of the user and the page
- **quality and trust**: penalties for spam, thin content, and untrustworthy sites

This section covers these content signals. Link structure, the strongest web-specific signal, gets its own section (Link Analysis: PageRank and HITS), and Learning to Rank shows how a search engine combines all signals into one ranking.

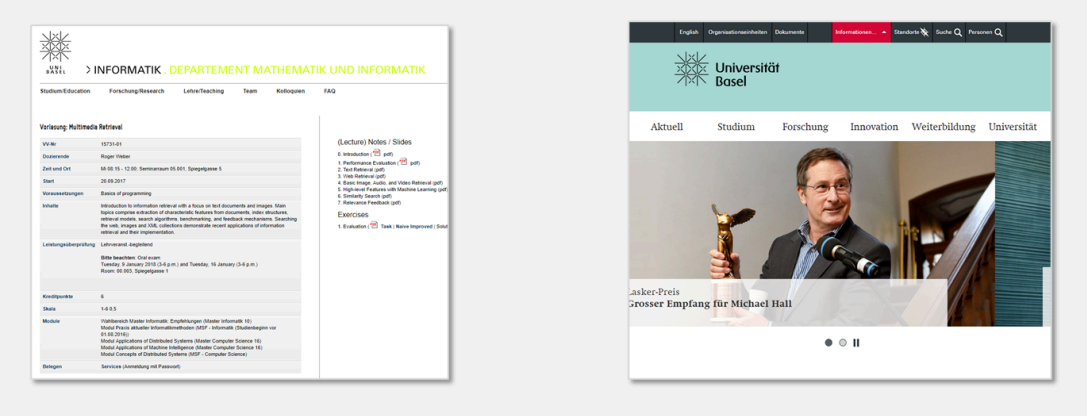


Figure 8.9: Two candidate pages for the query “uni basel”: a course page (left) and the university home page (right).

Term Proximity

The query “roger federer” is not a request for any page that contains “roger” and “federer”: a marathon result list with a Roger in row 12 and a Federer in row 4,000 matches both terms but answers nothing. In Chapter 3 we added frequent phrases such as “New York City” to the vocabulary as n-grams. That works for a fixed list of known phrases, but it cannot adapt to the new names and phrases that appear with every news event. Names matter a lot on the web, since people, places, and products are among the most frequent search topics. We therefore want a proximity measure that works for any combination of query terms and is computed at query time.

The index provides the raw material. Instead of storing only the term frequency in a posting, a **positional index** stores every position at which the term occurs in the document. This list is called a **hit list**. As document D, we take the lead section of the English Wikipedia article on Roger Federer (revision of 25 October 2026, CC BY-SA 4.0). We count positions in its plain text without the pronunciation guide; the lead is 626 words long. For the query “roger federer”, the hit lists are:

term	positions in D
“roger”	1, 532
“federer”	2, 57, 119, 165, 184, 235, 279, 337, 373, 419, 472, 509, 533, 560, 619

The article writes the full name only twice: in its first words, “Roger Federer (born 8 August 1981) is a Swiss former professional tennis player”, and near the end, in “the Roger Federer Foundation”. In between, it follows the usual style and refers to him by his surname alone.

At query time, we pair up occurrences of the two terms. The full cross product would create 30 pairs, most of them meaningless: “roger” at position 1 and “federer” at position 619 are more than 600 words apart. Instead, we merge both hit lists into one list sorted by position and form a pair wherever two neighbouring entries belong to different terms (Büttcher, Clarke, and Lushman; see the Further Reading section of the chapter summary). The merged list starts with roger (1), federer (2), federer (57) and ends with federer (509), roger (532), federer (533), federer (560), federer (619).

This gives three pairs: (1, 2), (509, 532), and (532, 533); position 532 belongs to two of them. The thirteen surname-only mentions in between form no pair, since a pair needs both terms. The pair (509, 532) is in reversed order and links the end of one mention to the start of the next: “Federer played an instrumental role in the creation of the Laver Cup team competition. He is also an active philanthropist, having established the Roger”. The rule treats both query terms alike, needs no window size, and works unchanged for queries with more than two terms.

Next, we sort the pair distances into bins, an idea that goes back to the original Google design of Brin and Page. Bin i counts the pairs at distance i, for $i = 1, \ldots, 10$. Pairs that are farther apart are “not even close” and do not count. The distances are 1, 23, and 1, so bin 1 holds two pairs, and the reversed pair at distance 23 drops out. Finally, a weight vector that decreases with distance turns the bin counts into a score.

Key Formula: Proximity Score

$$\text{score}_{\text{prox}}(Q, D) = \sum_{i=1}^{m} \text{bin}_i(Q, D) \cdot w_i \qquad \text{with } w_1 > w_2 > \ldots > w_m \tag{8.1}$$

Merge the hit lists, pair neighbouring hits of different query terms, count the pairs at each distance, and reward close pairs more than distant ones.

Example: Proximity score for “roger federer”

With the bin counts from above and the decreasing weights $w = (89, 55, 34, 21, 13, 8, 5, 3, 2, 1)$:

distance i	1	2	3	4	5	6	7	8	9	10
bin_i	2	0	0	0	0	0	0	0	0	0
w_i	89	55	34	21	13	8	5	3	2	1

$$\text{score}_{\text{prox}} = 2 \cdot 89 = 178 \tag{8.2}$$

The two pairs that count are the two places where the article writes the full name. The thirteen surname-only mentions earn nothing, a typical weakness of person search, since writers fall back to the surname.

Now take a marathon result list of 5,000 runners: a long table of ranks, names, and finishing times. Six runners have the first name Roger and six the surname Federer, among them a Roger in row 12 and a Federer in row 4,000. By term counts, the list is a serious candidate: it contains “roger” three times as often as the article. Merging the hit lists gives 11 pairs, more than the article’s 3. But every pair joins two different runners hundreds of rows apart, all pairs drop out of the bins, and the list scores 0. Proximity pushes the article above the list.

Production systems use more refined functions. They weight in-order pairs higher than reversed ones, treat a pair that crosses a sentence or paragraph boundary as weaker evidence, and often replace the bins by a smooth function of the distance d, such as $1/d^2$ in the method of Büttcher et al. Our example treats both orders alike and ignores boundaries. Here this makes no difference: the only reversed pair, which also crosses a sentence boundary, is too far apart to count. The idea stays the same: the closer the query terms appear, the more likely the page is about the combination and not just about each term on its own. Proximity has a storage price. The postings lists of Inverted Files store only the documents that contain a term; a positional index adds every position of the term and is therefore considerably larger. Web search engines pay this price, because the same positions also answer phrase queries such as “new york city” without indexing every n-gram.

Field Weighting with BM25F

HTML gives us more than a sequence of words. A term in the `<title>` was chosen by the author to describe the whole page. A term in an `<h1>` heading names a main topic, and a term in bold marks something the author wanted to stress. Classical retrieval counts all of these occurrences alike. Figure 8.10 shows the home page of the University of Basel with every occurrence of "university" and the tag that surrounds it.

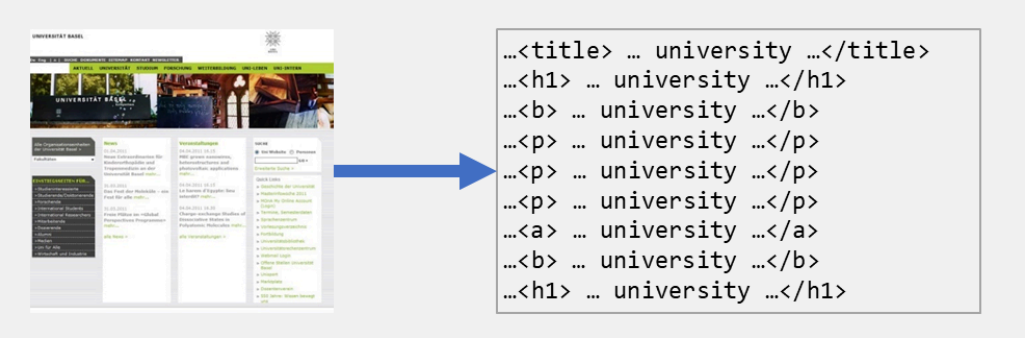


Figure 8.10: Occurrences of the term "university" on the University of Basel home page, with their enclosing HTML tags.

The page contains "university" nine times. Grouped by tag, we get one occurrence in the title, two in `<h1>` headings, two in bold text, three in paragraphs, and one in a link. A natural first idea is to saturate the count in each tag separately, as BM25 does for the whole document, and then add up the weighted results. This idea is flawed. If every field saturates on its own, a spammer can repeat a keyword in the title, in every heading, in bold text, and in the body, and collect the full saturated score once per field. The combination grows almost linearly with the number of fields, and the protection that saturation was meant to provide is gone.

BM25F (BM25 with fields) does it the other way around. It first combines the per-field counts into one weighted term frequency and only then applies the BM25 saturation:

Key Formula: BM25F

$$\tilde{\mathrm{tf}}(t, D) = \sum_{f} w_f \cdot \mathrm{tf}_f(t, D) \tag{8.3}$$

$$\mathrm{sim}_{\mathrm{BM25F}}(Q, D) = \sum_{t \in Q} \mathrm{idf}(t) \cdot \frac{\tilde{\mathrm{tf}}(t, D) \cdot (k_1 + 1)}{\tilde{\mathrm{tf}}(t, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\mathrm{avgdl}}\right)} \tag{8.4}$$

Weight each occurrence by its field (w_f for title, heading, body, …), add the weighted counts, then saturate once. A title occurrence counts like several body occurrences, but repetition in any field still runs into the same saturation limit.

The full model by Robertson, Zaragoza, and Taylor (see the Further Reading section of the chapter summary) normalizes the length of each field separately before combining. The simplified version above uses a single document length and is enough to see the effect.

Example: BM25F for "university"

We use the field weights title 13, `<h1>` 5, `<b>` 2, `<p>` 1, and `<a>` 0.5, with $k_1 = 1.2$ as in Chapter 1. To isolate the saturation effect, we switch off length normalization ($b = 0$) and leave out the IDF factor, which is the same for all pages.

field	title	h1	b	p	a
tf_f	1	2	2	3	1
$w_f \cdot \mathrm{tf}_f$	13	10	4	3	0.5

The weighted term frequency is $\tilde{\text{tf}} = 30.5$, and the saturated term score is $\frac{30.5 \cdot 2.2}{30.5 + 1.2} = 2.117$. Compare two other pages:

page	occurrences of “university”	$\tilde{\text{tf}}$	term score
University home page	9, spread over title, headings, body	30.5	2.117
Blog post	9, all in paragraphs	9	1.941
Keyword-stuffed page	40, all in paragraphs	40	2.136

Nine well-placed occurrences on the home page score almost as high as forty stuffed ones, and the stuffed page gains only 0.02 over the home page for 31 additional repetitions. Had we saturated each field separately and then added the weighted scores, the home page would score 24.7, and a page that repeats the term 10 times in each of title, `<h1>`, `<b>`, and `<p>` would collect 41.3 instead of 2.188.

Saturation matters more on the web than in a curated collection, because keyword stuffing is a standard SEO technique (The Web Search Landscape). Field weighting also extends beyond HTML tags: the URL, the domain name, and the anchor texts of incoming links (next subsection) are all fields with their own weights.

Anchor Text

The anchor text of a link is the clickable text that leads to another page. It usually describes the target page in a few precise words, chosen not by the target’s author but by someone else. Generic anchors such as “click here” or “more” carry no information, but with thousands of links to a page the useful descriptions dominate. Figure 8.11 shows two pages that link to the University of Basel home page: a Wikipedia list of Swiss universities and an Erasmus exchange portal. Both describe the target as “University of Basel” or “Universität Basel”.

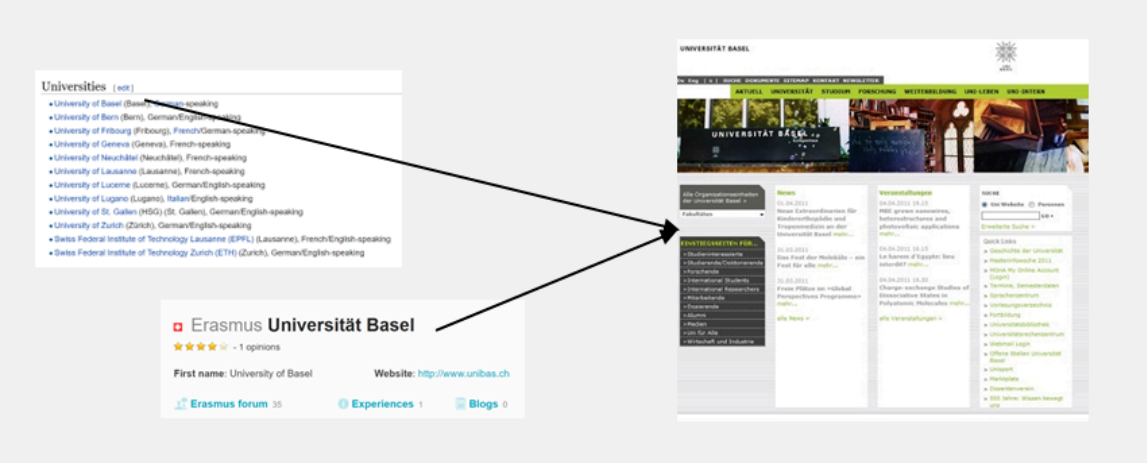


Figure 8.11: Two external pages (a Wikipedia list and an Erasmus portal) link to the University of Basel home page; their anchor texts describe the target.

Anchor text is valuable because it is hard to manipulate. A page author controls every word on their own page, but not the words that thousands of other sites use to link to it. Anchor text also describes pages that contain little text of their own, such as a home page made of images and navigation menus, and it can even describe pages the crawler has never fetched.

Using anchor text requires work at crawl time. When the crawler parses a page, it extracts each link together with its anchor text and stores the pair under the *target* URL (Crawling the Web). After the crawl, the search engine builds a separate **anchor index** in which each target page is described by the concatenation of all anchor texts pointing to it. At query time, the anchor index contributes its own score, usually with a higher weight than the page’s own text.

The saturation for anchor counts must be gentler than BM25's. Many independent pages voting for a description is exactly the evidence we want, so the score should keep growing with the number of votes, only more slowly. A logarithm such as $\log(1+n)$ for n anchor occurrences has this property.

An unbounded score invites manipulation. A **Google bomb** is a coordinated attempt to attach a keyword to a target page by placing that keyword in a very large number of links. The best-known example linked the phrase "miserable failure" to a biography of George W. Bush in 2003 and 2004 and pushed the page to the top for that query. The countermeasure is to count votes per domain: each domain has at most one vote for how well a term describes a target page.

Example: Per-domain anchor votes

The university home page receives 12,000 anchors containing "uni basel" from 900 different domains. A spammer places "uni basel" in 50,000 anchors on 3 domains, all pointing to their own page. With the score $\log(1+n)$:

counting	home page	spam page
all anchors, n = anchor count	$\log(1+12,000) = 9.39$	$\log(1+50,000) = 10.82$
one vote per domain, n = domain count	$\log(1+900) = 6.80$	$\log(1+3) = 1.39$

Counting all anchors, the spam page wins. Counting one vote per domain, the home page wins by a wide margin, and the spammer would need hundreds of independent domains to compete.

Location

Many queries have a local intent that the user does not spell out. Someone typing "best pizza" on a phone in Basel wants a restaurant within walking distance, not the best pizzeria in Naples. Google introduced a major local-search update, nicknamed Pigeon, in 2014, which tied local results more closely to the user's location and to the distance to businesses. Today, location influences the ranking of restaurants, shops, services, events, and many general queries.

Location-aware ranking needs three pieces of information:

1. **The location of a page.** For businesses, the owners often maintain an entry with address and opening hours in a directory such as Google Maps, so the search engine knows the location exactly. For other pages, the crawler estimates a location from the address on the page, the country domain, or the server's IP address.
2. **The location of the user.** The device may report its GPS position. Otherwise the search engine estimates the location from the IP address, which fails when the user connects through a VPN. If the query names a place, that place takes precedence: "best pizza in Paris" ranks Paris restaurants, even for a user in Basel. The query analyzer recognizes places as named entities (Query Understanding) and maps them to coordinates. This works for landmarks too, as in "best pizza near the Golden Gate Bridge".
3. **How much location matters for this query.** "best pizza" depends strongly on location. "Albert Einstein" is a global query, but a local museum with an Einstein exhibition can still receive a modest local boost.

Language

A user who asks for a recipe in German does not want English pages, even if an English page matches the query terms well. Language is therefore a signal of its own. The search engine detects the language of each page at indexing time, with the same classifiers that detect the language of a query (Chapter 3). On the user side, the language of the query, the interface language, the browser settings, and past queries indicate which languages the user reads.

The engine can use this signal in two ways. A classical filter restricts the results to pages in the user's languages, like the language option of an advanced search form. A filter is simple, but it is all or nothing: it also removes an English page that would be the best result when no good German page exists. Learning

to rank (Learning to Rank) instead treats the language match as one feature among many and learns from user behaviour how strongly to prefer pages in the query language. A recipe query then shows German pages first, while a query for an error message of a programming framework may still rank English pages high, because that is where the answers are.

Quality, Spam, and Trust

The Web Search Landscape described the kinds of low-quality content on the web, from outdated pages to scams and pages that exist only to rank. Search engines respond by estimating the quality of each page and the trustworthiness of its site. Typical measures include:

- **Penalties** for low-quality pages: thin content that wraps a few sentences in ads, pages copied from elsewhere, aggressive keyword stuffing, and clickbait whose content does not deliver what the title promises.
- **Removal** of entire sites from the results: link farms that exist only to create links, doorway pages that redirect users elsewhere, malware sites, and illegal content.
- **Site-level trust**: signals such as the known ownership of a site, its age, its editorial quality, and its reputation in other sources. A registered news organization or a university earns more trust than an anonymous site created last week.

Quality updates can shift results on a large scale. Google's 2011 Panda update, which targeted thin and low-quality content, noticeably affected about 12% of US English queries. Since then, quality assessment has moved to machine-learned classifiers. Google uses a spam detection system called SpamBrain, and its human quality raters judge pages by experience, expertise, authoritativeness, and trustworthiness (E-E-A-T); their judgments become training labels for the ranking models (Learning to Rank). Since 2023, large amounts of cheap, machine-generated text have made this defence harder and more important.

Quality signals have a cost. A ranking that rewards established, trusted sites makes it harder for a new site to reach the top, even if its content is excellent. Search engines therefore balance trust against freshness and diversity, a trade-off we meet again in Ranking Without a Query: Feeds.

Hands-on: Proximity, tags, and anchor text

Compute the "roger federer" proximity score on the Wikipedia article and a marathon result list, count term occurrences per HTML field and score them with BM25F, see why saturating each field separately rewards keyword stuffing, and defuse a Google bomb with one anchor vote per domain. Open notebook ->

Includes pre-run results: you can read through or download and experiment.

Link Analysis: PageRank and HITS

Return to the two candidate pages for "uni basel" in Figure 8.9. Content signals and anchor texts already favour the home page, but there is a stronger and more general clue: the structure of the web itself. Thousands of pages link to the university home page, while only a handful link to a single course page. A hyperlink is an implicit recommendation: the author of the linking page considered the target worth visiting. The web is a directed graph with pages as nodes and links as edges, and its topology tells us which pages the web itself considers important.

This section develops two classic ways to read that graph. **PageRank** computes one global importance score for every page, independent of any query. It was the key innovation of Google in 1998 and is still the reference model for ranking nodes in a graph. **HITS** distinguishes two roles that a page can play for a topic, hub or authority, and computes them on a small, query-specific subgraph. Both turn out to be the same mathematical idea: an eigenvector of a link matrix, computed by repeated matrix-vector multiplication.

Counting Links Is Not Enough

The simplest link-based score counts the incoming links of a page, its **in-degree**. Figure 8.12 shows a small web of seven pages. Page E receives six links and ranks first. B and G follow with four links each, C has two, A and D one each, and F none.

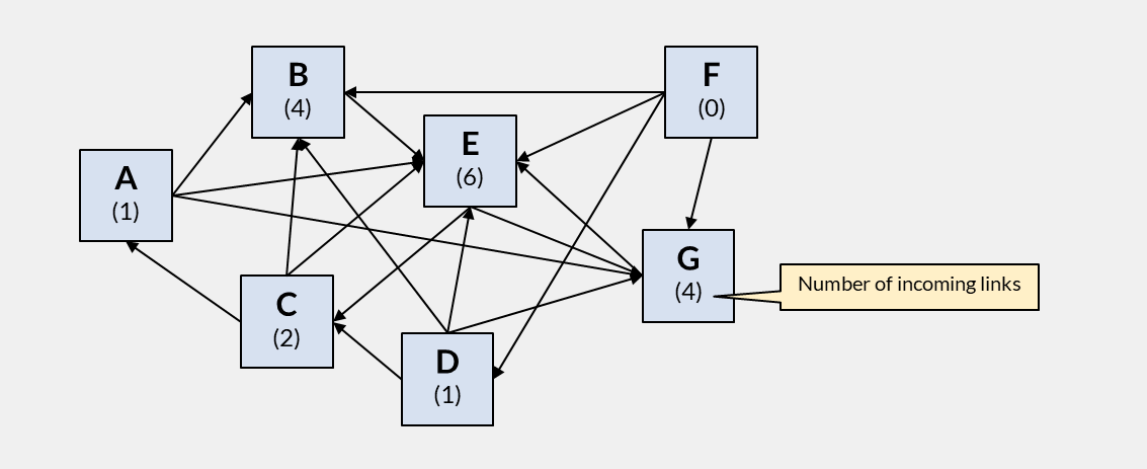


Figure 8.12: A seven-page web graph; the number below each page is its in-degree.

This ranking matches our first intuition: E is clearly the centre of this graph. But look at B and G, which tie with four incoming links. G receives a link from E, the most important page. B receives its links from A, C, D, and F, which are all minor pages themselves. G should rank above B, because a recommendation from an important page should count more than a recommendation from an unimportant one. The in-degree treats all links as equal votes.

There is a second, more practical problem. In-degree is easy to manipulate. Anyone can create a thousand pages that all link to a target page, a practice called **link farming**, and inflate its in-degree at almost no cost. We need a measure in which a link is worth only as much as the page it comes from. Then a thousand links from a thousand worthless pages add up to very little.

The Random Surfer

PageRank, developed by Larry Page and Sergey Brin at Stanford, defines importance through a thought experiment. A **random surfer** starts on some page and browses the web forever. On each page, the surfer does one of two things:

1. With probability α, the surfer clicks one of the links on the page, choosing each link with equal probability.
2. With probability $1 - \alpha$, the surfer gets bored and jumps to a random page, for example by typing a URL or opening a bookmark. Every page is equally likely as the jump target.

If a page has no outgoing links (a **dangling node**), the surfer always jumps to a random page. The PageRank of a page is the long-run fraction of time the surfer spends on it. In terms of probability theory, the surfer performs a Markov chain with pages as states, and PageRank is the stationary distribution of this chain. A page with high PageRank is a page that people browsing the web would visit often, which makes it a good candidate for the top of a result list.

To compute the PageRank of a page p, we ask how the surfer can arrive there. Either they follow a link from some page q that links to p (written $q \to p$), or they jump to p at random. Let $L(q)$ denote the number of outgoing links of q, and let $\mathbb{P}$ be the set of all pages with $N = \mid \mathbb{P} \mid$.

Key Formula: PageRank

$$\mathrm{PR}(p) = \frac{1-\alpha}{N} + \alpha \cdot \sum_{q \to p} \frac{\mathrm{PR}(q)}{L(q)} \qquad \forall p \in \mathbb{P} \tag{8.5}$$

A page is important if important pages link to it. Each page passes its PageRank in equal shares along its outgoing links; the random jump term $\frac{1-\alpha}{N}$ gives every page a small base score. A common choice is $\alpha = 0.85$.

The formula captures both problems with in-degree. A link from q contributes $\mathrm{PR}(q)$, so links from important pages count more. And the contribution is divided by $L(q)$, so a page that links to a thousand targets gives each of them only a thousandth of its vote. A link farm of a thousand pages that nobody links to has a combined PageRank of only about $1000 \cdot \frac{1-\alpha}{N}$, a negligible amount on a web of billions of pages. To inflate a target's PageRank, a spammer needs links from pages that already have a high PageRank, and those are much harder to get.

The random jump also matters. Without it, a group of pages that link only to each other, a **rank sink**, would absorb the surfer forever and collect all PageRank. The jump guarantees that the surfer can always leave, and it makes the Markov chain irreducible and aperiodic. These two properties guarantee that a unique stationary distribution exists.

Computing PageRank by Power Iteration

The PageRank formula defines one equation per page, and each equation refers to the PageRank of other pages. We solve this system iteratively. Let $\boldsymbol{r} \in \mathbb{R}^N$ be the vector of all PageRank values, and define the transition matrix $\boldsymbol{M} \in \mathbb{R}^{N \times N}$ as

$$M_{i,j} = \begin{cases} \frac{1}{L(p_j)} & \text{if } \; p_j \to p_i \\ \frac{1}{N} & \text{if } \; p_j \; \text{ has no outgoing links} \\ 0 & \text{otherwise} \end{cases} \tag{8.6}$$

Column j of $\boldsymbol{M}$ describes where the surfer goes from page p_j; every column sums to 1. With this matrix, the PageRank formula for all pages becomes

$$\boldsymbol{r} = \frac{1-\alpha}{N} \cdot \mathbf{1} + \alpha \cdot \boldsymbol{M}\boldsymbol{r} \tag{8.7}$$

where $\mathbf{1}$ is the column vector of N ones. **Power iteration** starts with the uniform distribution and applies the equation repeatedly:

- Initialization: $\boldsymbol{r}^{(0)} = \frac{1}{N} \cdot \mathbf{1}$
- Iteration: $\boldsymbol{r}^{(t+1)} = \frac{1-\alpha}{N} \cdot \mathbf{1} + \alpha \cdot \boldsymbol{M}\boldsymbol{r}^{(t)}$
- Stop when $\| \boldsymbol{r}^{(t+1)} - \boldsymbol{r}^{(t)} \|_1 < \varepsilon$

Each iteration is one multiplication of a sparse matrix with a vector. The web graph is extremely sparse: a page has a few dozen outgoing links, not billions. One iteration therefore costs time proportional to the number of links, and it parallelizes well across many machines. The number of iterations grows only slowly with the size of the graph. Page et al. reported that PageRank on a database of 322 million links converged in roughly 52 iterations, and on half of the data in roughly 45.

Example: PageRank on the seven-page web

We apply power iteration to the graph of Figure 8.12 with $\alpha = 0.85$ and $N = 7$. All pages start with $\frac{1}{7} = 0.143$. Page E receives links from A (3 outgoing links), B (1), C (3), D (4), F (4), and G (1). After the first iteration:

$$\mathrm{PR}^{(1)}(E) = \frac{0.15}{7} + 0.85 \cdot \frac{1}{7}\left(\frac{1}{3} + 1 + \frac{1}{3} + \frac{1}{4} + \frac{1}{4} + 1\right) = 0.021 + 0.85 \cdot 0.452 = 0.406 \quad (8.8)$$

Page F has no incoming links, so it only ever receives the random jump share $\frac{0.15}{7} = 0.021$. The iteration changes the vector by less than 10^{-4} (L1 norm) after 15 iterations and by less than 10^{-8} after 36. The final values:

page	A	B	C	D	E	F	G
in-degree	1	4	2	1	6	0	4
PageRank	7.4%	10.5%	18.6%	2.6%	37.5%	2.1%	21.2%

Figure 8.13 shows the same graph with node sizes proportional to PageRank. E is still the centre. G and C have overtaken B, even though C has only two incoming links. The reason is E: it has only two outgoing links, so C and G each receive $\frac{0.85}{2} = 42.5\%$ of E's PageRank. B's four incoming links come from pages that are themselves weak and that split their PageRank across three or four targets.

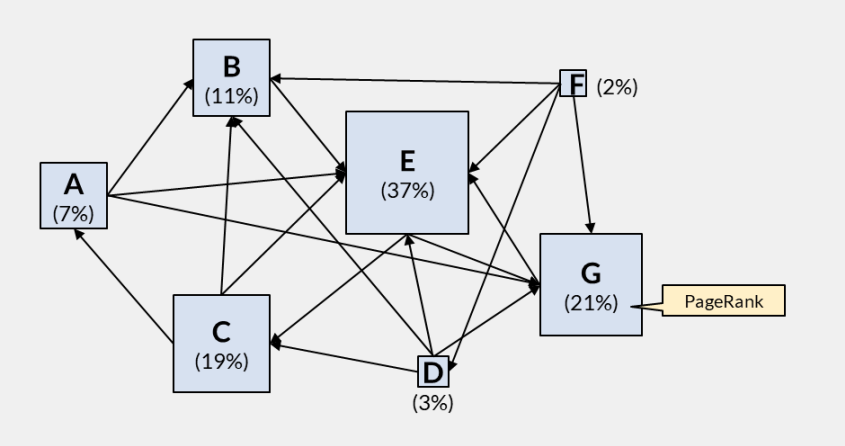


Figure 8.13: PageRank on the seven-page web graph with $\alpha = 0.85$; node size is proportional to PageRank.

The parameter α controls how far the surfer follows links before jumping. With a small α, the random jump dominates and all pages move towards the uniform value $\frac{1}{N}$. With α close to 1, the link structure dominates, weakly linked pages lose almost everything, and the iteration needs more steps to converge:

α	A	B	C	D	E	F	G	iterations to 10^{-8}
0.50	9.7%	13.2%	15.3%	8.0%	28.7%	7.1%	17.8%	17
0.85	7.4%	10.5%	18.6%	2.6%	37.5%	2.1%	21.2%	36
0.95	7.0%	9.6%	19.8%	0.9%	39.8%	0.7%	22.2%	47

PageRank is not a relevance score

PageRank measures how often a random surfer visits a page. It does not know the query. A search engine computes PageRank once after each crawl and uses it as one feature among many, combined with content signals such as BM25F and proximity. Ranking by PageRank alone would return the same famous pages for every query. PageRank is therefore not "how relevant a page is", but "how prominent a page is in the web graph".

PageRank also favours old pages. A new page, however good, has not yet collected incoming links, and links from important pages take time to appear. This is one reason why search engines add freshness signals for news and events, where the best page may be only a few minutes old.

Personalized PageRank

The random jump in PageRank picks every page with the same probability. Nothing forces this choice. If the surfer always jumps back to a fixed set of pages $\mathbb{S}$, the stationary distribution measures importance *from the point of view of* $\mathbb{S}$: pages that are close to $\mathbb{S}$ in the link graph and reachable through many paths receive high scores. This variant is called **Personalized PageRank**. Formally, we replace the uniform jump vector $\frac{1}{N} \cdot \mathbf{1}$ with a jump distribution $\boldsymbol{s}$ that is non-zero only on $\mathbb{S}$:

$$\boldsymbol{r} = (1 - \alpha) \cdot \boldsymbol{s} + \alpha \cdot \boldsymbol{M}\boldsymbol{r} \tag{8.9}$$

With $\mathbb{S}$ chosen as the pages of one topic, for example all pages that a web directory lists under “sports”, we obtain a topic-specific importance score. Haveliwala’s topic-sensitive PageRank precomputes one such vector per topic and mixes them at query time according to the topic of the query. With $\mathbb{S}$ chosen as a single node, for example one user in a social network, Personalized PageRank ranks all other nodes by how closely connected they are to that user. This is the basis for many “people you may know” and “who to follow” recommendations, which we revisit in Ranking Without a Query: Feeds.

Hubs and Authorities: HITS

In 1998, Jon Kleinberg proposed a different reading of the link graph. He observed that pages play two distinct roles for a topic, shown in Figure 8.14:

- An **authority** is a page that the community recognizes as a definitive source on a topic, such as Wikipedia, IMDb, or the official site of an organization. Good authorities receive many links.
- A **hub** is a page that collects links to the authorities of a topic without providing much content itself, such as a curated list of resources or a directory. Good hubs have many outgoing links.

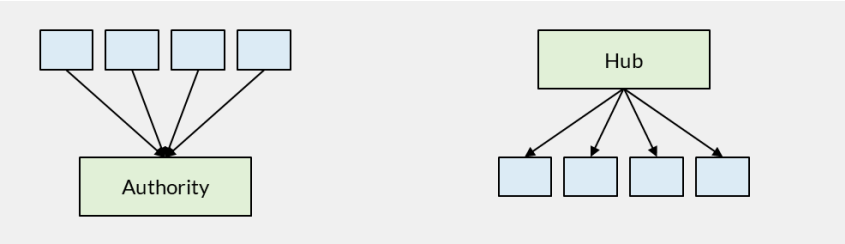


Figure 8.14: An authority receives links from many pages; a hub links to many pages.

The two roles reinforce each other. A good hub links to good authorities, and a good authority is linked from good hubs. Crucially, both are defined *for a topic*: a page that is a great authority on Basel is not an authority on jaguars. HITS (Hyperlink-Induced Topic Search) therefore does not run on the whole web, but on a small graph built for each query Q, as shown in Figure 8.15:

1. **Root set**: take the top results for Q from a text search engine, for example the top 200. This set already contains relevant pages, but not necessarily the best hubs and authorities.
2. **Base set**: extend the root set with all pages that a root page links to, and with pages that link to a root page (limited to, say, 50 per root page). Links within the same domain are usually dropped, since they are mostly navigation menus.

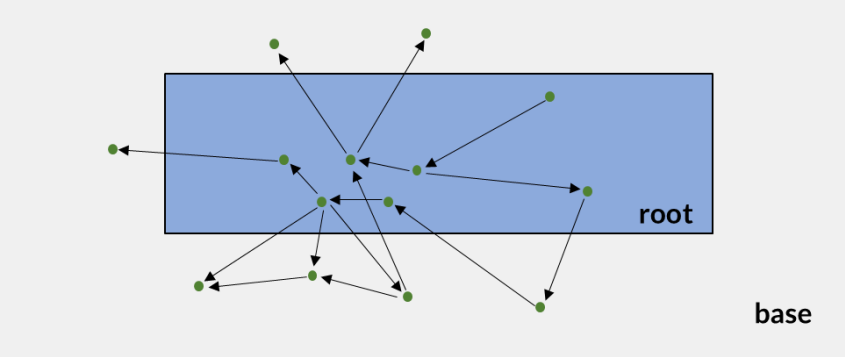


Figure 8.15: HITS expands the root set (the top text-search results) into the base set by following links in both directions.

The outgoing links of a root page are found by parsing it. The incoming links require a link index of the web, which a search engine builds during the crawl anyway. For each page p in the base set, HITS computes an authority value $a(p)$ and a hub value $h(p)$.

Key Formula: HITS Update

$$a^{(t+1)}(p) = \sum_{q \to p} h^{(t)}(q) \qquad h^{(t+1)}(p) = \sum_{p \to q} a^{(t)}(q) \tag{8.10}$$

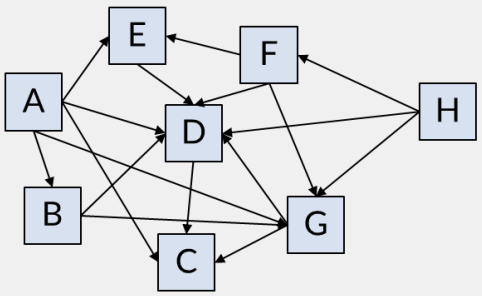
followed by normalization, so that $\sum_p a(p)^2 = \sum_p h(p)^2 = 1$. The authority of a page is the sum of the hub values of the pages that link to it; the hub value of a page is the sum of the authority values of the pages it links to.

The iteration starts with $a^{(0)}(p) = h^{(0)}(p) = \sqrt{1/N}$ for all N pages of the base set and stops when the values change by less than ε. In matrix notation, with the adjacency matrix $A_{i,j} = 1$ if $p_i \to p_j$ and 0 otherwise, the updates are $\boldsymbol{a}^{(t+1)} = \boldsymbol{A}^\top \boldsymbol{h}^{(t)}$ and $\boldsymbol{h}^{(t+1)} = \boldsymbol{A}\boldsymbol{a}^{(t)}$. Like PageRank, this is a power iteration: $\boldsymbol{a}$ converges to the principal eigenvector of $\boldsymbol{A}^\top \boldsymbol{A}$ and $\boldsymbol{h}$ to the principal eigenvector of $\boldsymbol{A}\boldsymbol{A}^\top$.

Example: HITS on an eight-page base set

Figure 8.16 shows a base set of eight pages, its adjacency matrix, and the converged hub and authority values (as fractions of the unit-length vectors, rounded).

- A is the best hub ($h = 0.60$): it links to D, G, and E, the three best authorities.
- D is the best authority ($a = 0.69$), linked by six pages, followed by G ($a = 0.54$).
- E ($a = 0.31$) is a slightly better authority than C ($a = 0.29$), even though E has two incoming links and C has three. E is linked by the two strong hubs A and F. C is linked by A, by the average hub G, and by the weak hub D.
- Roles mix. B is a weak authority and an average hub; G is a top authority but also has some hub value. C and H are pure types: C has no outgoing links, and nobody links to H.



Adjacency Matrix

	A	B	C	D	E	F	G	H		*h*	*a*
A		1	1	1	1		1			60%	0%
B				1			1			36%	18%
C										0%	29%
D			1							9%	69%
E				1						20%	31%
F				1	1		1			46%	12%
G			1	1						29%	54%
H				1		1	1			40%	0%

Figure 8.16: An eight-page base set, its adjacency matrix, and the converged hub (h) and authority (a) values.

PageRank and HITS answer different questions. PageRank is computed once, offline, for the whole web, and measures global prominence regardless of the query. HITS is computed at query time on a few thousand

pages and finds the best sources for one topic. This is also HITS's main weakness: building and iterating a base graph for every query is expensive, and the topic focus is fragile, since a few strongly linked off-topic pages in the base set can take over the result. Web search engines adopted PageRank-style global scores. The hub and authority idea survived elsewhere. SALSA, a random-walk variant of HITS by Lempel and Moran, powered Twitter's "Who to Follow" service, where the accounts closest to a user act as hubs and the accounts they follow act as authorities (Ranking Without a Query: Feeds).

SALSA: hubs and authorities as a random walk (optional reading)

HITS sums the hub values of all linking pages and then normalizes. A hub passes its full value to every page it links to, however many links it has, and the principal eigenvector concentrates on the most densely linked group of pages in the base set. Lempel and Moran called this the **tightly knit community effect**: a small group of pages that all link to each other can take over the result, even if the group is off topic. SALSA (Stochastic Approach for Link-Structure Analysis) replaces the sums with a random walk, in the spirit of PageRank.

The walk alternates between the two roles. From an authority p, the surfer follows one of the incoming links of p backwards to a hub, each with equal probability. From a hub q, the surfer follows one of the outgoing links of q forwards to an authority, again with equal probability. With $\text{in}(p)$ the number of incoming and $\text{out}(q)$ the number of outgoing links, the updates are

$$a^{(t+1)}(p) = \sum_{q \to p} \frac{h^{(t)}(q)}{\text{out}(q)} \qquad h^{(t+1)}(q) = \sum_{q \to p} \frac{a^{(t)}(p)}{\text{in}(p)} \tag{8.11}$$

Each hub splits its value among its outgoing links, and each authority splits its value among its incoming links. Both vectors remain probability distributions, so no normalization step is needed.

Lempel and Moran showed that the stationary distribution of this walk has a closed form. Within each connected component of the graph, the authority value of a page is proportional to its in-degree, and the hub value to its out-degree. A dense community therefore gains only as much as it has links, and the iteration is not even necessary. The price is the mutual reinforcement of HITS: within a component, a link from a strong hub counts as much as a link from a weak one. SALSA is in effect an in-degree count on the query-specific base set. Its topic focus comes from the base set, not from the link weights.

On the eight-page base set of Figure 8.16, all pages form one component with 17 links, so each authority value is its in-degree divided by 17: D 35%, G 24%, C 18%, E 12%, B and F 6% each. The hub values follow the out-degrees: A 29%, F and H 18% each, B and G 12% each, D and E 6% each. The top two authorities agree with HITS, but C and E swap places. HITS ranks E above C because the strong hubs A and F link to E. SALSA ignores hub strength and ranks C above E, since it has three incoming links and E only two.

Early experiments with HITS revealed three problems. An entire domain linking to one external page creates an overpowered authority. Automatically generated links, such as advertisements or "site by" credits, grant authority to off-topic pages. And for a query such as "jaguar car", the more frequent term dominates, so the top authorities are about cars in general rather than about Jaguar.

Making HITS robust: the Bharat and Henzinger improvements (optional reading)

Krishna Bharat and Monika Henzinger proposed two refinements of HITS in 1998 that address these problems.

The first refinement fixes overpowered hubs and authorities by letting each *domain* cast one vote instead of each page. If k pages of one domain link to a page p, each of these links receives the weight $aw(q, p) = \frac{1}{k}$ in the authority update of p. If a page p links to l pages of one domain, each of these links receives the weight $hw(p, q) = \frac{1}{l}$ in the hub update of p. Figure 8.17 shows the case of three pages on the example domain `a.com` linking to one page, and of one page linking to four pages on `b.com`. The updates become

$$a^{(t+1)}(p) = \sum_{q \to p} aw(q,p) \cdot h^{(t)}(q) \qquad h^{(t+1)}(p) = \sum_{p \to q} hw(p,q) \cdot a^{(t)}(q) \tag{8.12}$$

The second refinement keeps the computation on topic. Bharat and Henzinger build a reference document C from the terms with the highest $\mathrm{tf} \cdot \mathrm{idf}$ weights in the root set, so rare and specific terms such as "jaguar" are represented more strongly. For each page p in the base set, they compute the cosine similarity $s(p) = \frac{\boldsymbol{c}^{\top}\boldsymbol{p}}{\|\boldsymbol{c}\| \cdot \|\boldsymbol{p}\|}$ between the $\mathrm{tf} \cdot \mathrm{idf}$ vectors of C and p (Vector Space Retrieval). Pages with a similarity below a threshold, for example the median of all $s(p)$ values, are removed from the base set, which drops about half of the pages.

The remaining similarity values weight the propagation, so that relevant pages pass on more of their hub and authority value:

$$a^{(t+1)}(p) = \sum_{q \to p} aw(q,p) \cdot s(q) \cdot h^{(t)}(q) \qquad h^{(t+1)}(p) = \sum_{p \to q} hw(p,q) \cdot s(q) \cdot a^{(t)}(q) \tag{8.13}$$

In their evaluation on 28 topics, the combination of both refinements improved the precision of the top ten results by at least 45% over the original HITS algorithm. The same two ideas, counting votes per domain and weighting links by topical similarity, reappear in many later link-analysis methods, including the per-domain anchor votes of Ranking Signals Beyond Term Counts.

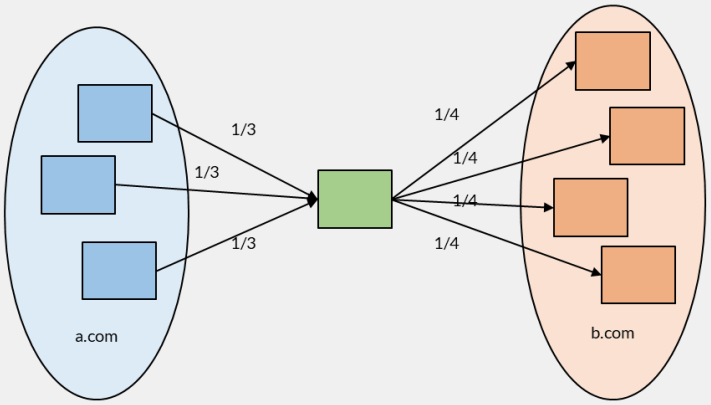


Figure 8.17: Domain weights: three pages of `a.com` share one vote ($\frac{1}{3}$ each); four linked pages of `b.com` share one vote ($\frac{1}{4}$ each).

Hands-on: PageRank and HITS

Let a random surfer loose on the seven-page web, compute PageRank by power iteration, vary the damping factor α, create a rank sink, and personalize the random jump. Then compute HITS hubs and authorities on the eight-page base set, check them against the eigenvectors, and compare with SALSA. Open notebook ->

Includes pre-run results: you can read through or download and experiment.

Learning to Rank

The previous two sections produced a long list of signals: BM25F over several fields, proximity, anchor text, PageRank, site quality, location, and language. A real search engine adds many more, such as freshness, the URL depth, the share of users who clicked a page for similar queries, or the score of a neural reranker. Google has long described its ranking as built from hundreds of signals. All of them must be combined into one score per page, and the combination decides what the user sees.

This section starts with an example that combines four signals by hand and asks how to find the best combination. It then formalizes the question as a machine learning problem with three parts: the training data, the scoring model, and the loss. Finally, it shows how the models are trained and how search engines keep collecting the labels for training, mainly from the clicks of their users. The section closes with feeds, which use the same machinery to rank content without any query.

The Ranking Stack

No search engine applies its most expensive ranking function to billions of pages. Ranking happens in stages, and each stage hands a smaller candidate set to a more expensive model, as Figure 8.18 shows. The retriever uses the inverted index (4 - Index for Text Retrieval) and a cheap score such as BM25F to select on the order of ten thousand candidates. For each candidate, the system computes the full feature vector. A learned ranker, typically an ensemble of gradient-boosted decision trees, orders the candidates. A neural reranker, such as a cross-encoder (Semantic Similarity with Transformers), rescores the top hundred. Finally, the result page is composed from the ranked list and other elements such as ads and knowledge panels (The Web Search Landscape).

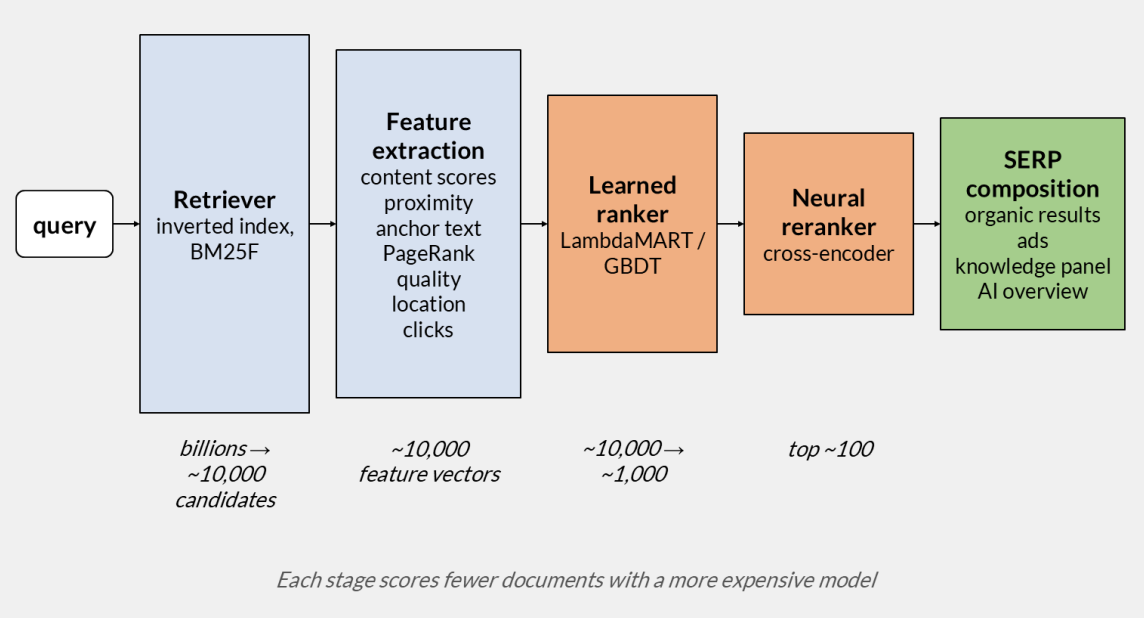


Figure 8.18: The web ranking stack: each stage passes fewer candidates to a more expensive model.

This section is about the learned ranker in the middle of the stack. Its input is one feature vector per query-page pair, and its output is a score used for sorting.

Combining Signals by Hand

Recall the query "uni basel" from the chapter opener. We take four candidate pages and four of the signals from the previous sections, already scaled to comparable ranges. Each page also has a relevance grade from 0 (irrelevant) to 3 (perfect), as in the graded benchmarks of Evaluating Graded Relevance:

page	BM25F	proximity	anchor	PageRank	grade y
d_1: University of Basel home page	2.1	0.9	6.8	0.9	3
d_2: a course page	2.3	0.8	1.1	0.2	1

page	BM25F	proximity	anchor	PageRank	grade y
d_3: a marathon result list	1.9	0.1	0.0	0.1	0
d_4: Wikipedia article	2.0	1.0	4.5	0.7	2

The ideal ranking is d_1, d_4, d_2, d_3. Ranking by BM25F alone gives d_2, d_1, d_4, d_3, because the course page is short and dense with the query terms. With the DCG of Evaluating Graded Relevance, the ideal ranking has $\text{IDCG} = \frac{3}{1} + \frac{2}{\log_2 3} + \frac{1}{2} + \frac{0}{\log_2 5} = 4.762$, and the BM25F ranking reaches $\text{nDCG} = 0.817$.

The simplest way to use more signals is a weighted sum: multiply each signal by a weight and add up the results. We try a few weight vectors $\boldsymbol{w}$ by hand, in the order (BM25F, proximity, anchor, PageRank):

weights $\boldsymbol{w}$	scores d_1, d_2, d_3, d_4	ranking	nDCG
$(1, 0, 0, 0)$	2.10, 2.30, 1.90, 2.00	d_2, d_1, d_4, d_3	0.817
$(1, 0.5, 0, 0)$	2.55, 2.70, 1.95, 2.50	d_2, d_1, d_4, d_3	0.817
$(1, 0, 0, 1)$	3.00, 2.50, 2.00, 2.70	d_1, d_4, d_2, d_3	1
$(1, 0.5, 0.3, 1)$	5.49, 3.23, 2.05, 4.55	d_1, d_4, d_2, d_3	1
$(0, 0, 1, 0)$	6.80, 1.10, 0.00, 4.50	d_1, d_4, d_2, d_3	1

Adding proximity does not help, since the three pages that matter score almost the same on it. Adding PageRank or anchor text fixes the ranking. But three quite different weight vectors are now perfect, including one that uses the anchor text alone. This query cannot tell them apart. Other queries can: anchor text alone fails for a new page that nobody links to yet, and a high PageRank weight pushes strong home pages to the top even for queries that they match only weakly.

A good weight vector must therefore work on average over thousands of queries at once. It must cover hundreds of signals, not four, and it must be chosen again whenever the signals or the web change. Tuning by hand does not scale to this. Instead, we collect queries with judged candidates and let an algorithm search for the weights that rank them best. **Learning to Rank** (LTR) treats the combination of signals as a supervised machine learning problem: given examples of queries, candidate pages, and their known relevance, learn a function that orders the candidates well.

The Learning-to-Rank Problem

Figure 8.19 shows the three parts of the problem. A **scoring model** s_θ maps the feature vector $\boldsymbol{x}$ of a query-page pair to a score s, and sorting by s gives the ranked list. The parameters θ of the model are not set by hand. They are learned from **training data**, a set of queries with candidate pages and labels y. The **loss** $\mathcal{L}$ compares the scores with the labels and measures how well the model ranks: the better the ranking, the smaller the loss. Training updates θ until the loss is small, that is, until sorting by s orders the candidates like their labels.

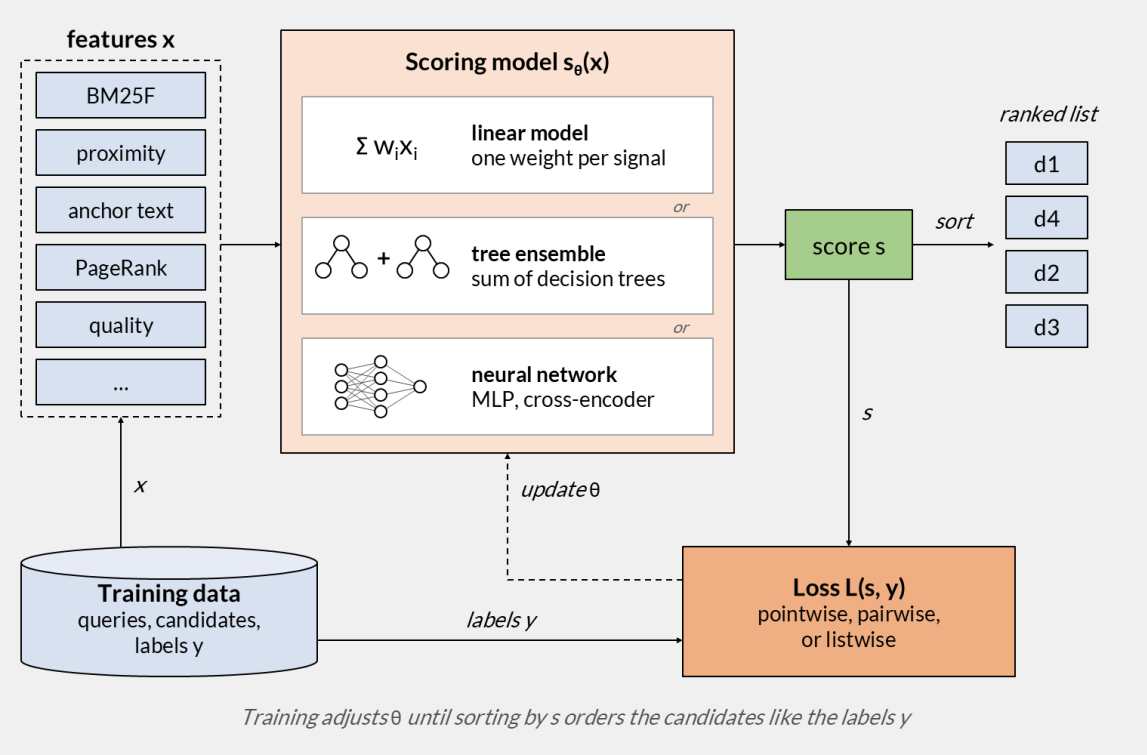


Figure 8.19: The learning-to-rank problem: a scoring model maps features to a score, and training updates its parameters θ from a loss on labeled data.

Since only the order of the candidates of one query matters, the loss is computed per query and summed over all training queries:

Key Formula: The Learning-to-Rank Objective

$$\theta^* = \arg\min_\theta \sum_Q \mathcal{L}\left(\, \boldsymbol{s}_Q, \boldsymbol{y}_Q \,\right) \qquad \boldsymbol{s}_Q = \left(\, s_\theta\left(\boldsymbol{x}_{Q,D_1}\right), \ldots, s_\theta\left(\boldsymbol{x}_{Q,D_n}\right) \,\right) \tag{8.14}$$

Find the parameters whose scores order the candidates $D_1, \ldots, D_n$ of every training query Q as their labels $\boldsymbol{y}_Q$ do. The loss $\mathcal{L}$ measures, per query, how far the scores disagree with the labels.

In the running example, the training data holds one query with $n = 4$ candidates and $\boldsymbol{y}_Q = (3, 1, 0, 2)$. The model is the weighted sum, its parameters θ are the four weights $\boldsymbol{w}$, and every row of the table above is one candidate θ. The three parts of the objective can be chosen independently, and the following subsections take them in turn.

Training Data: Features and Labels

Each training example is a pair (Q, D) of a query and a candidate page, described by a feature vector $\boldsymbol{x}_{Q,D}$ and a label $y_{Q,D}$. The training set is $\mathbb{T} = \left\{\left(Q, \boldsymbol{x}_{Q,D}, y_{Q,D}\right)\right\}$ over many queries, each with tens to hundreds of judged candidates. Features fall into three groups:

- **Query-dependent features** describe how well the page matches the query: BM25F per field, proximity, anchor-text score, cosine similarity of embeddings (Building Semantic Search).
- **Query-independent features** describe the page itself: PageRank, site quality, page length, URL depth, age.
- **Query features** describe the query alone: its length, its intent class, whether it names a place. They do not separate the candidates of one query, but they let the model weight other features differently for different kinds of queries.

The label $y_{Q,D}$ says how relevant D is for Q. The classic source is human judgment on a graded scale, as in our table. Search engines employ thousands of trained raters who judge pages according to detailed

guidelines. The second source is user behaviour, mainly clicks. Both sources are discussed in Labels for Continuous Training.

The Scoring Model

The scoring model decides which combinations of signals the ranker can express at all. Three model families are common, shown in the middle of Figure 8.19.

A **linear model** computes the weighted sum of our hand-tuned attempts, $s_\theta(\boldsymbol{x}) = \boldsymbol{w}^\top \boldsymbol{x}$ with $\theta = \boldsymbol{w}$. It has one weight per signal, is cheap to evaluate, and its weights are easy to read. But each signal adds its contribution regardless of the other signals. A linear model cannot express that a high PageRank should count only if the page also matches the query well. Without that condition, a strong home page outranks better pages for every query that it matches even weakly.

A **regression tree** captures such interactions. It asks a sequence of threshold questions about the features and returns the value stored in the leaf it reaches. A tree with three questions reproduces the grades of our four pages exactly. It first asks whether the anchor score exceeds 3. If so, it asks whether PageRank exceeds 0.8 and returns 3 (d_1) or 2 (d_4). If not, it asks whether proximity exceeds 0.5 and returns 1 (d_2) or 0 (d_3). PageRank is used only for pages with strong anchor text, which is an interaction that no single weight can express. A single tree is coarse, so rankers use a **tree ensemble** that adds up the outputs of hundreds of small trees, $s_\theta(\boldsymbol{x}) = \sum_{m=1}^{M} f_m(\boldsymbol{x})$. In gradient-boosted decision trees (GBDT), each new tree f_m is trained to correct the remaining errors of the trees before it. Trees also handle features on different scales and missing values without manual feature engineering. They do need features that mean the same for every query, since one threshold such as "BM25F > 5" applies to all queries alike. A query with rare terms produces high BM25F scores because of their IDF, and a query with frequent terms produces low ones, so a score of 5 can be mediocre for the first and excellent for the second. Learning-to-rank benchmarks therefore scale each query-dependent feature to the range 0 to 1 within the candidates of each query.

A **neural network** learns the interactions in its hidden layers. A small multi-layer perceptron computes $s_\theta(\boldsymbol{x}) = \boldsymbol{w}_2^\top \mathrm{ReLU}(\boldsymbol{W}_1 \boldsymbol{x} + \boldsymbol{b}_1)$, with all weight matrices and vectors in θ. A cross-encoder (Semantic Similarity with Transformers) goes one step further: it reads the text of the query and the page directly instead of a feature vector, and learns its own features.

The three families trade expressiveness against cost and transparency. Tree ensembles are the strongest choice for hand-crafted features such as those in our table, and they sit in the middle of the ranking stack. Cross-encoders are the most expressive, but also the most expensive, so they only rescore the top candidates.

The Loss: Three Ways to Learn a Ranking

The loss decides what the training optimizes. LTR methods differ in what they treat as one training example and in the loss they minimize. Figure 8.20 compares the three families on the same four documents.

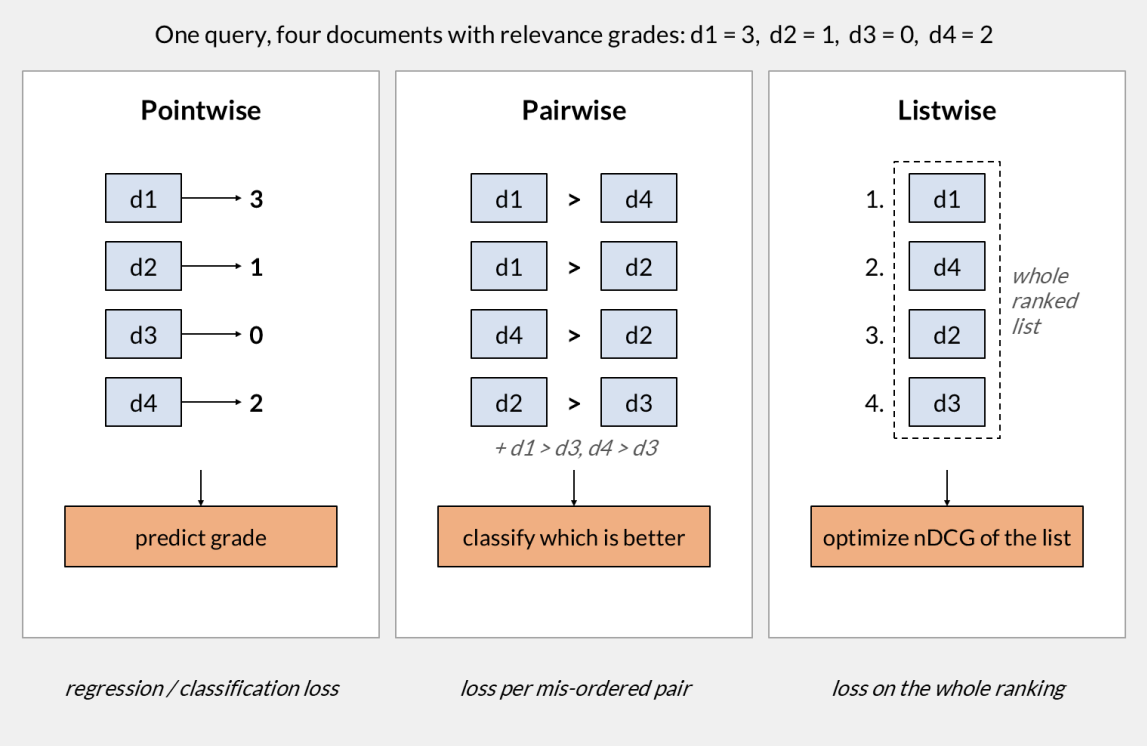


Figure 8.20: Pointwise, pairwise, and listwise learning on one query with four graded documents.

Pointwise: Predict the Grade

The pointwise approach ignores that we want a ranking. It treats each (Q, D) pair on its own and trains a regression model to predict the grade y, or a classifier to predict the grade class. Any standard learner works, for example least-squares regression with the loss $\mathcal{L} = \sum_D \big(s_\theta(\boldsymbol{x}_{Q,D}) - y_{Q,D} \big)^2$. On our four pages, a linear model reproduces the grades 3, 1, 0, 2 exactly, but with the weights $(0.13, 0.65, 0.73, -3.16)$. A negative PageRank weight makes no sense: four examples and four free weights can fit any grades, so a perfect fit on so little data says nothing about which signals matter.

The pointwise view has two weaknesses. First, it spends effort on the wrong goal. Predicting 4.0 instead of 3 for d_1 costs a squared error of 1.0, but it is harmless, since d_1 stays on top. Predicting 1.6 for d_4 and 1.7 for d_2 costs only $0.16 + 0.49 = 0.65$, yet it swaps two pages. Second, grades are not comparable across queries. For a query with one perfect page, a grade of 2 may be the second-best result. For a query with fifty perfect pages, the same grade lands on page three. Only the order *within* a query matters.

Pairwise: Order Each Pair

The pairwise approach turns ranking into classification of pairs. For each query, every pair of documents with different grades becomes one example: d_i should rank above d_j, written $d_i \succ d_j$, if $y_i > y_j$. Our four pages yield six such pairs: $d_1 \succ d_2$, $d_1 \succ d_3$, $d_1 \succ d_4$, $d_2 \succ d_3$, $d_4 \succ d_2$, and $d_4 \succ d_3$. A model that orders every pair correctly produces the ideal ranking.

RankNet (Burges et al., 2005) is the classic pairwise method. It turns the score difference into a probability that d_i should rank above d_j, using the logistic function $\sigma(z) = \frac{1}{1+e^{-z}}$, and minimizes the cross-entropy loss on all pairs:

Key Formula: Pairwise Logistic Loss (RankNet)

$$P(d_i \succ d_j) = \sigma\left(s_i - s_j \right) \qquad \mathcal{L} = \sum_{(i,j):\, y_i > y_j} -\log \sigma\left(s_i - s_j \right) \tag{8.15}$$

A pair costs little if the better document scores clearly higher, and a lot if it scores lower. Training pushes the scores of each pair apart in the right direction.

Example: Pairwise loss of two scoring functions

With BM25F alone, the scores are $s = (2.1, 2.3, 1.9, 2.0)$. Two of the six pairs are in the wrong order: $d_1 \succ d_2$ (2.1 < 2.3) and $d_4 \succ d_2$ (2.0 < 2.3). Because all scores lie close together, even the correct pairs have probabilities near 0.5. The total loss is $\mathcal{L} = 4.052$.

With the linear model $\boldsymbol{w} = (1.0, 0.5, 0.3, 1.0)$, the scores are $(5.49, 3.23, 2.05, 4.55)$. All six pairs are in the right order, and the loss drops to $\mathcal{L} = 1.044$. Training moves the weights from the first situation towards the second, as Training the Model shows.

The pairwise view is much closer to the ranking goal, and it produces many training examples from few judgments. Clicks also naturally produce pairs, as we will see. Its weakness is that every pair counts the same. Swapping the pages at ranks 1 and 2 hurts the user far more than swapping the pages at ranks 40 and 41, but both cost the same in $\mathcal{L}$.

Listwise: Optimize the Ranking Measure

The listwise approach optimizes a measure of the whole ranked list, such as nDCG. The difficulty is that nDCG depends on the sorted order, so it changes in jumps when two scores cross and has no useful gradient. **LambdaRank** (listwise, built on RankNet's pairs) solves this with a simple trick. It keeps the pairwise gradients of RankNet, but scales the gradient of each pair by how much nDCG would change if the two documents swapped positions in the current ranking:

Key Formula: LambdaRank Pair Weight

$$\lambda_{ij} = \left|\, \Delta\mathrm{nDCG}_{ij} \,\right| \cdot \sigma\left(-\left(s_i - s_j\right)\right) \tag{8.16}$$

The push on a pair grows with the damage a swap would do to nDCG and with how badly the pair is currently ordered. Pairs near the top of the list, where nDCG is most sensitive, get the strongest updates.

Example: Lambda weights for the BM25F ranking

The current ranking by BM25F is d_2, d_1, d_4, d_3. For each of the six pairs, we swap the two documents in this ranking and measure the change in nDCG:

pair	current ranks	$\lvert\,\Delta$ nDCG$\rvert$	$\sigma(-(s_i - s_j))$	λ_{ij}
$d_1 \succ d_2$	2, 1	0.155	0.550	0.085
$d_4 \succ d_2$	3, 1	0.105	0.574	0.060
$d_1 \succ d_3$	2, 4	0.126	0.450	0.057
$d_2 \succ d_3$	1, 4	0.120	0.401	0.048
$d_4 \succ d_3$	3, 4	0.029	0.475	0.014
$d_1 \succ d_4$	2, 3	0.027	0.475	0.013

The strongest push goes to $d_1 \succ d_2$, the mis-ordered pair at the top of the list. The correctly ordered pair $d_1 \succ d_4$ at ranks 2 and 3 gets the weakest push, since swapping it would cost little. RankNet (pairwise comparison only) would weight all six pairs by the σ column alone, regardless of where the pair sits in the list. LambdaRank (listwise) multiplies by the $\lvert\,\Delta$ nDCG$\rvert$ column and thereby focuses on the top of the list.

Pointwise, pairwise, listwise: what is the training example?

The three families are not model types. Any of the scoring models above (a linear function, trees, or a neural network) can be trained with any of the three losses. What differs is the unit of training and the

loss: one document and its grade (pointwise), one pair and its order (pairwise), or one whole result list and its nDCG (listwise).

Learning to Rank in Practice

The formalization leaves two practical questions. How does an algorithm find the parameters θ^* for a given model and loss? And where do the labels come from, not once, but continuously, as the web and its users change?

Training the Model

For a linear model or a neural network, training uses gradient descent, as for the word embeddings of Word Embeddings. For a linear model with the RankNet loss, each pair pulls the weights in the direction of the feature difference $\boldsymbol{x}_i - \boldsymbol{x}_j$ between the better and the worse page. The pull is strong if the pair is currently mis-ordered and weak if it is already well separated:

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{w}} = - \sum_{(i,j):\, y_i > y_j} \sigma\left(-\left(s_i - s_j\right)\right)\left(\boldsymbol{x}_i - \boldsymbol{x}_j\right) \qquad \boldsymbol{w} \leftarrow \boldsymbol{w} - \eta \frac{\partial \mathcal{L}}{\partial \boldsymbol{w}} \tag{8.17}$$

Example: One gradient step from BM25F alone

We start with $\boldsymbol{w} = (1, 0, 0, 0)$, the BM25F ranking with $\mathcal{L} = 4.052$. Summed over the six pairs, the gradient is $(-0.06, -1.19, -11.82, -1.45)$. Its largest component is the anchor weight, because the anchor scores differ most between the better and the worse pages of each pair. One step with the learning rate $\eta = 0.05$ gives $\boldsymbol{w} = (1.003, 0.060, 0.591, 0.073)$ and the scores $(6.24, 3.02, 1.92, 4.78)$. The ranking is now ideal, and the loss drops to $\mathcal{L} = 0.762$. Further steps keep pushing the correct pairs apart: after 50 steps, the loss is 0.115, although the ranking no longer changes.

With thousands of queries, the gradient sums the pulls of all their pairs, and the weights settle on a compromise that ranks well on average. LambdaRank uses the same update, but multiplies the term of each pair by its $|\ \Delta\mathrm{nDCG}_{ij}\ |$, which is exactly the lambda weight of the example above.

Trees cannot be trained by gradient descent on their parameters, since their thresholds and leaf values do not change smoothly. **LambdaMART** (LambdaRank trained with decision trees) combines the lambda weights with MART (multiple additive regression trees), a form of gradient-boosted decision trees. LambdaRank defines what to optimize: the lambdas. MART defines how the model grows: each new tree is fitted to the lambdas of the current ensemble, so the ensemble grows in the direction that improves nDCG most. LambdaMART won the Yahoo! Learning to Rank Challenge in 2010 and remains a strong baseline for ranking over many hand-crafted features. Libraries such as LightGBM and XGBoost implement it with a `lambdarank` objective.

Gradient-boosted trees over hand-crafted features dominated web ranking in the 2000s and early 2010s. Since 2019, transformer models have taken over a growing part of the stack. Google announced in 2019 that it uses BERT to understand queries and rank results, affecting about one in ten English queries in the US at launch. A cross-encoder reads query and page together and captures meaning that no feature vector expresses, for example that “can you get medicine for someone pharmacy” is about picking up a prescription for another person. Neural rankers have not replaced learning to rank; they have joined it. A neural reranker is trained with the same pointwise, pairwise, or listwise losses, often on the same mix of human labels and clicks. Its output can become one more feature for a tree ensemble, or the final stage of the stack in Figure 8.18.

Example: Learning to rank on TREC-COVID

The TREC-COVID collection of Approximate Nearest-Neighbor Search has 50 topics with 66,336 graded judgments. We take the judged papers of each topic as its candidate list and describe each topic-paper pair by eight features: BM25 on title and abstract, BM25F, proximity, the fraction of query terms present,

the two field lengths, and the cosine similarity of sentence embeddings as a neural score. Each ranker is trained on 40 topics and evaluated on the other 10, in five rounds, with features normalized per topic:

ranker	nDCG@10
BM25F alone (no learning)	0.660
neural score alone (no learning)	0.704
linear, pointwise (least squares)	0.765
linear, pairwise (RankNet)	0.791
trees, pointwise (regression)	0.807
trees, listwise (LambdaMART)	0.816
LambdaMART without the neural score	0.649

Every ranker that learns from all eight features beats the best single signal. Within each model family, the pairwise or listwise loss beats the pointwise one, and LambdaMART is best overall. Without per-topic normalization, the trees lose this lead (LambdaMART 0.789). The last row shows that the neural score does not replace the term statistics but completes them: without it, the remaining features are variants of the same term counts and add nothing over BM25F. On 50 topics, differences of about 0.01 are within the noise.

Labels for Continuous Training

A ranker is never trained only once. New pages appear, events create new queries, the meaning of queries shifts, and every new signal requires a new model. Search engines therefore retrain their rankers regularly, and they need a steady supply of fresh labels. Two sources provide them.

Human raters judge query-page pairs on a graded scale, following detailed guidelines. Their labels are precise and reflect the intended notion of quality, but they are slow and expensive. Search engines therefore spend them where precision matters most: on test sets that evaluate every new ranker offline with nDCG (When Evaluation Misleads), and on hard or new kinds of queries.

User behaviour is the second source: which results users click, how long they stay on the clicked page, and whether they return to the result list to try another result. Clicks are abundant and free, and they reflect the needs of real users rather than the opinion of raters. Every day of traffic adds millions of new examples. They are also heavily biased.

The strongest bias is **position bias**. Users click the top results much more often, partly because they are better, but mostly because they are on top. In an eye-tracking study, Joachims et al. swapped the first two results without telling the users, and the result at rank 1 still received most clicks. A click on rank 1 is weak evidence of relevance; a click on rank 5 is strong evidence, since the user read past four other results to get there. Joachims's early solution used clicks to create pairwise preferences: if the user skipped the result at rank 2 and clicked the one at rank 3, then $d_3 \succ d_2$ for this query. These pairs feed directly into a pairwise learner such as RankNet.

Modern **unbiased learning to rank** corrects position bias explicitly. It estimates the probability p_k that a user even looks at rank k, the propensity, for example through small randomized swaps in a fraction of the result pages. Each click at rank k is then weighted by $\frac{1}{p_k}$ in the loss, so clicks at rarely viewed positions count more. Under the assumption that users click a result exactly when they see it and find it relevant, this inverse propensity weighting removes the position bias in expectation.

Clicks are not relevance

A click shows that a result looked promising in the result list, not that the page satisfied the user. Attractive titles and thumbnails earn clicks regardless of quality, which rewards clickbait. Search engines therefore use refined signals: long dwell time on the clicked page counts as success, while a

quick return to the result list (a "pogo-stick" back to the results) counts as failure. Even so, a model trained on clicks optimizes what users click, which is not always what is good for them. We meet this tension again in Ranking Without a Query: Feeds.

Clicks also decide whether a new ranker goes live. Offline metrics on a judged test set cannot capture everything. Search engines therefore run online experiments: an **A/B test** shows the new ranker to a small fraction of users and compares click and satisfaction metrics with the current one. **Interleaving** mixes the results of both rankers into one list and checks which ranker's results attract more clicks; it needs far fewer users than an A/B test to detect a difference.

Together, these steps form a loop. The deployed ranker produces result pages, the result pages produce clicks, the debiased clicks train the next ranker, and online experiments decide whether it replaces the current one. The loop has a blind spot: users can only click what the ranker shows, so a page that never reaches the first result page never collects the clicks it would need to get there. The randomized swaps that estimate propensities also help here, since they show some results at positions where the current ranker would not put them. The same staged design and the same loop also power recommendation feeds, which we turn to next.

Hands-on: Learning to rank

Run the "uni basel" example through pointwise, pairwise (RankNet), and listwise (LambdaRank) learning, remove the position bias of simulated clicks, then train LambdaMART with LightGBM on 66,000 TREC-COVID judgments and watch a neural score join the content signals as one more feature. Open notebook ->

Includes pre-run results: you can read through or download and experiment.

Ranking Without a Query: Feeds

Open TikTok, YouTube, Instagram, or X, and the app shows content before the user types anything. Every swipe asks the same question as a search engine: of the hundreds of millions of items available right now, which few should this user see next? The difference is where the information need comes from. In web search, the user *pulls* information with a query. In a feed, the system *pushes* content, and the user's history, social connections, and context take the role of the query. For videos, music, and images, this has become the main way people discover content.

A feed reuses the machinery of this chapter and of earlier chapters almost unchanged, in the same staged design as the web ranking stack (Figure 8.21):

- **Candidate generation** takes the place of the retriever. One source is embedding retrieval: a **two-tower model** encodes the user and every item into one vector space. This is the bi-encoder of Semantic Similarity with Transformers with the user in the place of the query, and an approximate nearest-neighbour index (Approximate Nearest-Neighbor Search) returns the closest items in milliseconds. Since the item vectors are computed from the content, a new video can be recommended before anyone has watched it; the multimedia features of later chapters feed exactly this step. A second source is the social graph: Personalized PageRank and SALSA (Link Analysis: PageRank and HITS) find the accounts closest to a user, which yields "who to follow" suggestions and the posts that these accounts liked.
- **Ranking** is learning to rank. A heavy ranker predicts, for each candidate, the probability of every possible reaction: like, reply, share, watch to the end, hide, or report. This is pointwise learning to rank with several labels at once, and the predictions are combined in a weighted sum.
- **Filters** remove content the user has already seen or blocked, limit the number of posts per author, and mix in new creators and topics.

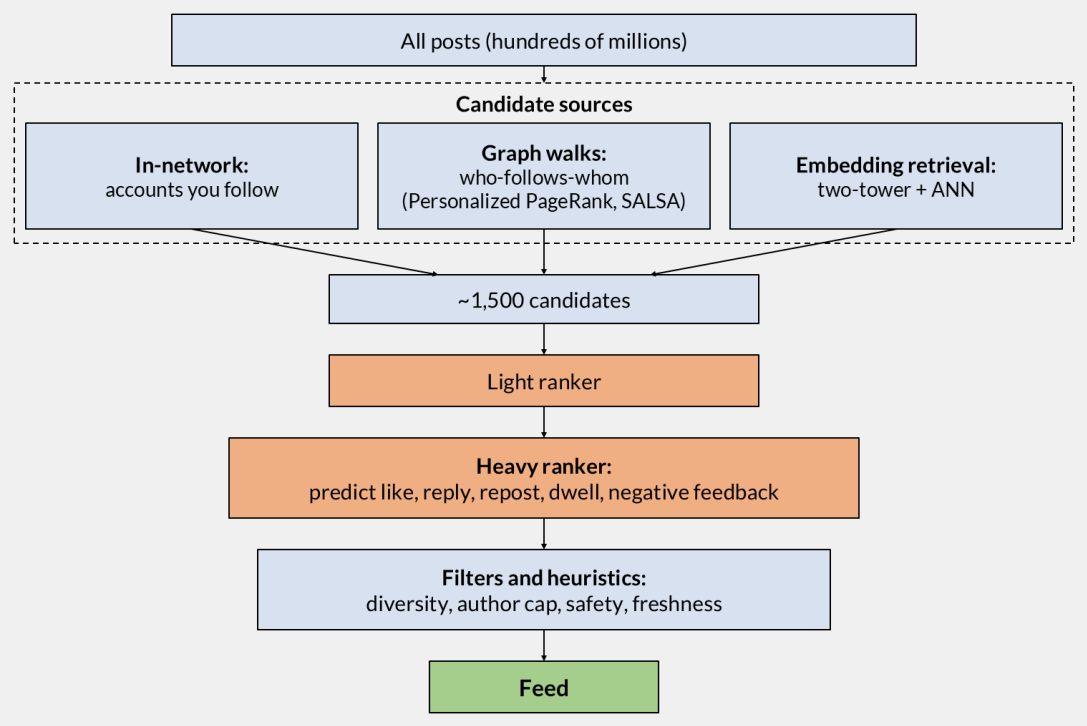


Figure 8.21: The recommendation funnel: candidate sources select about 1,500 posts, rankers order them, and filters shape the final feed.

The weights of the sum are a product decision, not a learned quantity, and they decide what the feed becomes. X, which published the source code of its home timeline in 2023, weighted a like with 0.5, a reply with 13.5, and a reply that the post's author engages with 75. Such weights favour conversations; high weights on shares or watch time favour content that spreads. The warning about clicks applies here with full force. Replies, shares, and watch time correlate with interest, but outrage and sensational content produce them as well. A feed trained purely on engagement can amplify divisive content and narrow what a user sees into a **filter bubble**. Platforms counter this with negative feedback signals, quality classifiers, diversity rules, and surveys that ask users directly whether a post was worth their time.

Feeds also face sharper versions of the evaluation problems of web search. The log only shows reactions to what the current system chose to show, so a feed reserves part of its slots for **exploration**, such as new creators and new topics, to learn anything new about its users. And since there is no ground truth for which post a user should have seen, feeds are evaluated almost entirely online, with A/B tests that track long-term effects such as whether users return next week, not only short-term engagement. Search and feeds thus share the same end point: both learn to rank from user behaviour, and both must keep the measured behaviour from replacing the goal it stands for.

Summary

Model Comparison

model	input	computed	scope	output	typical use
In-degree	link graph	offline	global	count of incoming links	naive baseline; easy to manipulate
PageRank	link graph	offline, once per crawl	global, query-independent	stationary visit probability	page-level prominence feature
Personalized PageRank	link graph + jump set	offline or per user	relative to a node or topic	proximity-weighted importance	topic-sensitive ranking, who to follow, feed candidates
HITS	query-specific base set	at query time	one topic	hub and authority values	topic distillation; hub/authority roles (SALSA)
Pointwise LTR	feature vector, grade	training offline	per document	predicted grade	simple baseline; multi-task engagement prediction in feeds
Pairwise LTR (RankNet)	pairs of documents	training offline	per pair	score difference	learning from click preferences
Listwise LTR (LambdaMART)	whole result list	training offline	per query	score optimized for nDCG	the learned ranker in web search

The link-analysis models all compute an eigenvector of a link matrix by power iteration; they differ in where the random walk restarts (everywhere, at a fixed set, or nowhere) and in which graph they walk (the whole web or a query-specific base set). The learning-to-rank families can use the same model and differ in the unit of training: one document, one pair, or one list. A modern search engine uses link analysis to produce features and learning to rank to combine them, and a recommendation feed reuses both: random walks and embeddings to generate candidates, and a learned multi-task ranker to order them.

Key Takeaways

1. Web search differs from classical retrieval in an uncurated, adversarial collection, short and ambiguous queries, huge result sets with nearly equal term scores, and a link graph that reveals which pages people consider important. The engine serves the intent most users mean, composes the result page for that intent, and relies on a crawler that collects the evidence about each page that the page itself does not provide.
2. Content signals go beyond term counts: proximity rewards query terms that appear close together, BM25F weights fields before a single saturation, anchor text describes a page in other people's words, counted with one vote per domain, and location, language, and site quality adapt the ranking to the user and the source.
3. PageRank is the stationary distribution of a random surfer and measures global prominence, not relevance; HITS computes topic-specific hubs and authorities on a query-dependent subgraph.

4. Learning to rank combines hundreds of signals; pairwise and listwise objectives target the order of results, and LambdaRank weights each pair by its effect on nDCG. Clicks are abundant training labels but must be corrected for position bias.
5. Feeds rank without a query, with the same staged design: embeddings and the social graph supply candidates, a learned ranker predicts the user's reactions, and the weights on these predictions decide what the platform optimizes.

Key Formulas

Key Formula: PageRank

$$\mathrm{PR}(p) = \frac{1-\alpha}{N} + \alpha \cdot \sum_{q \to p} \frac{\mathrm{PR}(q)}{L(q)} \tag{8.18}$$

A page is important if important pages link to it; each page splits its PageRank evenly among its outgoing links, and the random jump gives every page a base share.

Key Formula: HITS Update

$$a(p) = \sum_{q \to p} h(q) \qquad h(p) = \sum_{p \to q} a(q) \qquad \text{then normalize} \tag{8.19}$$

Good authorities are linked by good hubs; good hubs link to good authorities.

Key Formula: LambdaRank Pair Weight

$$\lambda_{ij} = \left| \, \Delta\mathrm{nDCG}_{ij} \, \right| \cdot \sigma \left(-\left(s_i - s_j \right) \right) \tag{8.20}$$

The RankNet gradient of a pair, scaled by how much swapping the pair would change nDCG.

Exam focus

Beyond the formulas above, be able to explain:

- Why in-degree is a poor and manipulable importance measure, and how PageRank fixes both problems (weighting by source importance, dividing by out-degree).
- The role of the damping factor α and the random jump: dangling nodes, rank sinks, uniqueness of the solution, and the effect on convergence.
- The difference between PageRank and HITS: global versus query-specific, offline versus at query time, one score versus two roles.
- Why BM25F saturates after combining fields, and why anchor votes are counted per domain.
- The difference between pointwise, pairwise, and listwise learning to rank, and why position bias makes raw clicks unreliable labels.
- How a feed mirrors the web ranking stack, and why optimizing engagement is not the same as optimizing value.

Self-Check Questions

1. (Understand) Explain the random surfer model. What does the PageRank of a page represent, and why is it not a measure of relevance to a query?
2. (Apply) Compute one power-iteration step of PageRank for page C in Figure 8.12, starting from the uniform distribution with $\alpha = 0.85$.
3. (Analyze) In the HITS example, E has two incoming links and C has three, yet E has the higher authority value. Explain why.
4. (Analyze) A spammer repeats a keyword ten times in the title, every heading, and the body of a page. Compare the effect under per-field saturation and under BM25F.
5. (Evaluate) A new ranker improves nDCG on the judged test set but reduces the click-through rate in an A/B test. Which result do you trust, and what further evidence would you collect?

6. (Evaluate) A short-video platform raises the weight of shares in its engagement score. Discuss the likely effects on what users see, and propose one safeguard.

Test Your Knowledge

Take the Chapter 8 Quiz ->

Further Reading

- Brin, S., & Page, L. (1998). **The anatomy of a large-scale hypertextual web search engine**. *Computer Networks and ISDN Systems*, 30(1-7), 107-117. PDF (archived). The original Google paper: crawler, index, anchor text, proximity hit lists, and PageRank in one system.
- Page, L., Brin, S., Motwani, R., & Winograd, T. (1999). **The PageRank citation ranking: Bringing order to the web**. Technical Report 1999-66, Stanford InfoLab. PDF. Defines PageRank, the random surfer, and power iteration, with the convergence results quoted in Link Analysis: PageRank and HITS.
- Kleinberg, J. M. (1999). **Authoritative sources in a hyperlinked environment**. *Journal of the ACM*, 46(5), 604-632. PDF. Introduces hubs, authorities, and the HITS iteration.
- Bharat, K., & Henzinger, M. R. (1998). **Improved algorithms for topic distillation in a hyperlinked environment**. *Proceedings of SIGIR 1998*, 104-111. PDF. The domain weights and similarity weighting of the HITS dropdown.
- Lempel, R., & Moran, S. (2001). **SALSA: The stochastic approach for link-structure analysis**. *ACM Transactions on Information Systems*, 19(2), 131-160. The random-walk variant of HITS in the optional reading of Link Analysis: PageRank and HITS, including the tightly knit community effect and the in-degree closed form.
- Haveliwala, T. H. (2002). **Topic-sensitive PageRank**. *Proceedings of WWW 2002*, 517-526. PDF (archived). Personalized PageRank with one precomputed vector per topic.
- Büttcher, S., Clarke, C. L. A., & Lushman, B. (2006). **Term proximity scoring for ad-hoc retrieval on very large text collections**. *Proceedings of SIGIR 2006*, 621-622. DOI. Pairs neighbouring hits of different query terms in the merged hit list and adds a proximity score to BM25; the pairing rule of Ranking Signals Beyond Term Counts.
- Robertson, S., Zaragoza, H., & Taylor, M. (2004). **Simple BM25 extension to multiple weighted fields**. *Proceedings of CIKM 2004*, 42-49. Argues for combining field counts before saturation and defines BM25F. A free treatment is in Robertson and Zaragoza's survey **The Probabilistic Relevance Framework: BM25 and Beyond** (*Foundations and Trends in Information Retrieval*, 3(4), 2009), listed in the Chapter 4 summary.
- Broder, A. (2002). **A taxonomy of web search**. *ACM SIGIR Forum*, 36(2), 3-10. PDF. The navigational, informational, and transactional query types.
- Olston, C., & Najork, M. (2010). **Web crawling**. *Foundations and Trends in Information Retrieval*, 4(3), 175-246. PDF. A survey of crawler architecture, politeness, freshness, and duplicate detection.
- Liu, T.-Y. (2009). **Learning to rank for information retrieval**. *Foundations and Trends in Information Retrieval*, 3(3), 225-331. PDF (archived). The standard survey of pointwise, pairwise, and listwise methods.
- Burges, C., Shaked, T., Renshaw, E., Lazier, A., Deeds, M., Hamilton, N., & Hullender, G. (2005). **Learning to rank using gradient descent**. *Proceedings of ICML 2005*, 89-96. PDF. Introduces RankNet and the pairwise logistic loss of Learning to Rank.
- Burges, C. J. C. (2010). **From RankNet to LambdaRank to LambdaMART: An overview**. Microsoft Research Technical Report MSR-TR-2010-82. PDF. The pairwise loss, the lambda weights, and boosted trees, explained by their author.
- Chapelle, O., & Chang, Y. (2011). **Yahoo! Learning to Rank Challenge overview**. *JMLR Workshop and Conference Proceedings*, 14, 1-24. PDF. The 2010 challenge on real web-search features; the winning team of track 1 combined twelve rankers, eight of them LambdaMART models.
- Joachims, T. (2002). **Optimizing search engines using clickthrough data**. *Proceedings of KDD 2002*, 133-142. PDF. Turns clicks into pairwise preferences for learning to rank.
- Joachims, T., Granka, L., Pan, B., Hembrooke, H., & Gay, G. (2005). **Accurately interpreting clickthrough data as implicit feedback**. *Proceedings of SIGIR 2005*, 154-161. PDF. The eye-tracking study behind position bias: clicks are informative but biased.

- Joachims, T., Swaminathan, A., & Schnabel, T. (2017). **Unbiased learning-to-rank with biased feedback**. *Proceedings of WSDM 2017*, 781-789. arXiv. Inverse propensity weighting to remove position bias from click data.
- Covington, P., Adams, J., & Sargin, E. (2016). **Deep neural networks for YouTube recommendations**. *Proceedings of RecSys 2016*, 191-198. PDF. The two-stage candidate generation and ranking design of a large video feed.
- Yi, X., Yang, J., Hong, L., Cheng, D. Z., Heldt, L., Kumthekar, A., Zhao, Z., Wei, L., & Chi, E. (2019). **Sampling-bias-corrected neural modeling for large corpus item recommendations**. *Proceedings of RecSys 2019*, 269-277. PDF. Training two-tower retrieval models with in-batch negatives at YouTube scale.
- Gupta, P., Goel, A., Lin, J., Sharma, A., Wang, D., & Zadeh, R. (2013). **WTF: The who to follow service at Twitter**. *Proceedings of WWW 2013*, 505-514. PDF. The circle of trust (Personalized PageRank) and SALSA on the follow graph.
- Aggarwal, P., Murahari, V., Rajpurohit, T., Kalyan, A., Narasimhan, K., & Deshpande, A. (2024). **GEO: Generative engine optimization**. *Proceedings of KDD 2024*, 5-16. arXiv. How page content can be tuned to be cited in generated answers.
- Twitter Engineering (2023). **Twitter's recommendation algorithm**. Blog post (archived), source code, and heavy ranker weights. The candidate sources, heavy ranker, and filters of a production feed.